

华中科技大学

计算机系统基础

题目： 计算机系统基础实验报告

专业班级 智科 2402

学 号 U202490035

姓 名 阮云轩

指导教师 赵进

报告日期 2025 年 12 月 2 日

目 录

1	引言	1
2	数据表示和等效运算	2
2.1	实验目的与要求	2
2.2	实验内容	2
2.3	实验记录及问题回答	4
2.4	体会	10
3	汇编和 C 的混合编程及优化	12
3.1	实验目的与要求	12
3.2	实验内容	12
3.3	算法设计	13
3.4	体会	28
4	机器级语言理解	29
4.1	实验目的与要求	29
4.2	实验内容	29
4.3	实验记录及问题回答	30
4.4	体会	36
5	缓冲区溢出攻击	37
5.1	实验目的与要求	37
5.2	实验内容	37
5.3	实验记录及问题回答	38
5.4	体会	41
6	链接炸弹拆除	42
6.1	实验目的与要求	42
6.2	实验内容	42
6.3	实验记录及问题回答	43
6.4	体会	53
7	ARM 指令系统的理解	54
7.1	实验目的与要求	54
7.2	实验内容	54

华中科技大学课程实验报告

7.3	实验记录及问题回答.....	54
7.4	体会.....	58
8	附录.....	59
8.1	实验一源码.....	59
8.2	实验二源码.....	69

1 引言

本实验旨在深入探究计算器系统的底层运作机制，从高级语言的抽象层下沉至内存与位操作的具体实现层。在现代计算器科学中，理解数据在内存中的物理布局以及 CPU 处理数据的逻辑门电路原理，是编写高效、健壮代码的基石。

通过实验任务，我们将直观地观测计算器内存的各种现象，模拟 CPU 的 ALU（算术逻辑单元）行为，这不仅是对逻辑思维的锻炼，更是对计算器二进制本质的探究。通过 GDB 调试工具的动态追踪，我们得以「打开黑盒」，亲眼见证数据在寄存器与内存间流转的微观过程。

2 数据表示和等效运算

2.1 实验目的与要求

- (1) 熟练掌握程序开发的基本方法，包括程序的编译、链接和调试；
- (2) 熟悉数据的表示形式；
- (3) 熟悉地址的计算方法、地址的内存转换；
- (4) 用常见的按位操作（移位、按位与/或/非/异或）等实现运算表达式的等效运算。

2.2 实验内容

任务 1 数据压缩

定义了结构 `student`，以及结构数组变量 `old_s[N]`, `new_s[N]`; ($N = 5$)

```
struct student {  
    char name[8];  
    short age;  
    float score;  
    char remark[200]; // 备注信息  
};
```

编写程序，将 `old_s[N]` 中的所有信息依次紧凑 (压缩) 存放到一个字符数组 `message` 中，然后从 `message` 解压缩到结构数组 `new_s[N]` 中。打印压缩前 (`old_s`)、解压后 (`new_s`) 的结果，以及压缩前、压缩后存放数据的长度。

要求：

1. 输入的第 0 个人姓名 (`name`) 为自己的名字，分数为学号的最后两位；
2. 编写指定接口的函数完成数据压缩。压缩函数有两个：

```
int pack_student_bytebybyte(student* s, int sno, char *buf);  
int pack_student_whole(student* s, int sno, char *buf);
```

`s` 为待压缩数组的起始地址；`sno` 为压缩人数；`buf` 为压缩存储区的首地址；
两个函数的返回均是调用函数压缩后的字节数。`pack_student_bytebybyte`

要求一个字节一个字节的向 *buf* 中写数据；*pack_student_whole* 要求对 *short*、*float* 字段都只能用一条语句整体写入，用 *strcpy* 实现串的写入。

3. 使用指定方式调用压缩函数：*old_s* 数组的前 $N1$ ($N1 = 2$) 个记录压缩调用 *pack_student_bytebybyte* 完成；后 $N2$ ($N2 = 3$) 个记录压缩调用 *pack_student_whole*，两种压缩函数都只调用 1 次。

4. 使用指定的函数完成数据的解压。解压函数的格式：

```
int restore_student(char *buf, int len, student* s);
```

buf 为压缩区域存储区的首地址；*len* 为 *buf* 中存放数据的长度；*s* 为存放解压数据的结构数组的起始地址；返回解压的人数。解压时不允许使用函数接口之外的信息（即不允许定义其他全局变量）。

5. 仿照调试时看到的内存数据，以十六进制的形式，输出 *message* 的前 40 个字节的內容，并与调试时在内存窗口观察到的 *message* 的前 40 个字节比较是否一致。
6. 对于第 0 个学生的 *score*，根据浮点数的编码规则指出其各部分的编码，并与观察到的内存表示比较，验证是否一致。
7. 指出结构数组中各元素的存放规律，指出字符串数组、*short* 类型的数、*float* 型的数的存放规律。

任务 2 编写位运算程序

按照要求完成给定的功能，并自动判断程序的运行结果是否正确。（从逻辑电路与门、或门、非门等等角度，实现 CPU 的常见功能。所谓自动判断，即用简单的方式实现指定功能，并判断两个函数的输出是否相同。）

1. *int absVal(int x)*; 返回 *x* 的绝对值

仅使用 *!*、*~*、*&*、*^*、*|*、*+*、*<<*、*>>*，运算次数不超过 10 次。

判断函数：*int absVal_standard(int x)* { return (*x* < 0) ? -*x* : *x*;}

2. *int negate(int x)*; 不使用负号，实现 $-x$

判断函数：*int negate_standard(int x)* { return -*x*;}

3. *int bitAnd(int x, int y)*; 仅使用 *~* 和 *|*，实现 *&*

判断函数：*int bitAnd_standard(int x, int y)* { return *x* & *y*;}

4. *int bitOr(int x, int y)*; 仅使用 *~* 和 *&*，实现 *|*

5. *int bitXor(int x, int y)*; 仅使用 *~* 和 *&*，实现 *^*

6. `int isTmax(int x)`; 判断 x 是否为最大的正整数 (7FFFFFFF),
只能使用 `!`、`~`、`&`、`^`、`|`、`+`
7. `int bitCount(int x)`; 统计 x 的二进制表示中 1 的个数
只能使用 `! ~ & ^ | + << >>`, 运算次数不超过 40 次
8. `int bitMask(int highbit, int lowbit)`; 产生从 `lowbit` 到 `highbit` 全为 1,
其他位为 0 的数。例如 `bitMask(5,3) = 0x38`; 要求只使用 `! ~ & ^ | + << >>`;
运算次数不超过 16 次。
9. `int addOK(int x, int y)`; 当 $x + y$ 会产生溢出时返回 1, 否则返回 0
仅使用 `!`、`~`、`&`、`^`、`|`、`+`、`<<`、`>>`, 运算次数不超过 20 次
10. `int byteSwap(int x, int n, int m)`; 将 x 的第 n 个字节与第 m 个字节
交换, 返回交换后的结果。 n 、 m 的取值在 0~3 之间。
例: `byteSwap(0x12345678, 1, 3) = 0x56341278`
`byteSwap(0xDEADBEEF, 0, 2) = 0xDEEFBEAD`
仅使用 `!`、`~`、`&`、`^`、`|`、`+`、`<<`、`>>`, 运算次数不超过 25 次
11. `int bang(int x)`; 当 $x = 0$ 时, 返回 1; 其他情况返回 0。实现逻辑非 (!)
例: `bang(3) = 0`; `bang(0) = 1`;
仅使用 `~`、`&`、`^`、`|`、`+`、`<<`、`>>`, 运算次数不超过 12 次
提示: 只有当 $x = 0$ 时, x 与 $-x$ 的最高二进制位会同时为 0。
12. `int bitParity(int x)`; 当 x 有奇数个二进制位 0, 返回 1; 否则返回 0
例: `bitParity(5) = 0`; `bitParity(7) = 1`;
仅使用 `!`、`~`、`&`、`^`、`|`、`+`、`<<`、`>>`, 运算次数不超过 20 次。
提示: 只有当 x 的高字与低字的对应位 (第 0 位对应第 16 位, 第 1 位对应
第 17 位, 依次类推) 同时为 1, 则出现了成对的二进制位 1, 此时, 可以将
对应的二进制位置为 0, 不会影响二进制位 1 的个数的奇偶性判断。

2.3 实验记录及问题回答

2.3.1 任务 1 数据存放的压缩与解压编程

1) 分析

首先分析待压缩的数据结构, `student` 结构体中的姓名 (`name`) 和备注 (`remark`) 分别固定占用 8 和 200 个字节, 但实际输入的姓名或备注往往很短, 这导致内

存中存在大量用于占位的无效 0x00 字节，这就是主要的压缩空间；其次是年龄 (age)，虽然 short 类型占 2 字节，但人的年龄通常不超过 255，高位字节几乎恒为 0，也可以省去；只有分数 (score) 因遵循 IEEE 754 浮点数标准，结构复杂且无冗余，无法简单压缩。在 GDB 调试中观察内存发现，输入的中文姓名呈现 UTF-8 编码（如 e5 bd ad...），且年龄 18 在内存中显示为 12 00，验证了 x86 架构的小端序存储模式。压缩前，这些有效数据被大片的 0 隔开，呈现稀疏状态；压缩后，通过指针操作去除了所有填充的 0，名字后紧接分隔符和年龄的低位字节，数据变得非常紧凑，证明了压缩算法有效消除了空间浪费。

测试范例

阮云轩 18 35 goodgoodstudydaydayup

Kit 197 70.5 Smart

Kingsley 3 21.5 Macao

Ryan 14 66.6 Rich

Kevin 20 2.33 Hust

然后我们输入数据后看下内存空间 (2-1)，可以观察到，我们的空间利用率是十分低效的，有很多空内存的位置，于是我们需要想办法去设计算法去压缩我们的内存空间，提高利用率。

```
(gdb) x/100xb &old_s
0x7ff669268040 <old_s>: 0xe9 0x98 0xae 0xe9 0x9b 0xb2 0xe8 0xbb
0x7ff669268040 <old_s>: 0xe9 0x98 0xae 0xe9 0x9b 0xb2 0xe8 0xbb
0x7ff669268048 <old_s+8>: 0x12 0x00 0x00 0x00 0x00 0xc 0x42 0x67 0x6f
0x7ff669268050 <old_s+16>: 0x6f 0x64 0x67 0x6f 0x6f 0x64 0x73 0x74
0x7ff669268058 <old_s+24>: 0x75 0x64 0x79 0x64 0x61 0x79 0x64 0x61
0x7ff669268060 <old_s+32>: 0x79 0x75 0x70 0x00 0x00 0x00 0x00 0x00
0x7ff669268068 <old_s+40>: 0x00 0x00 0x00 0x00 0x00 0x00 0x00 0x00
0x7ff669268070 <old_s+48>: 0x00 0x00 0x00 0x00 0x00 0x00 0x00 0x00
0x7ff669268078 <old_s+56>: 0x00 0x00 0x00 0x00 0x00 0x00 0x00 0x00
0x7ff669268080 <old_s+64>: 0x00 0x00 0x00 0x00 0x00 0x00 0x00 0x00
0x7ff669268088 <old_s+72>: 0x00 0x00 0x00 0x00 0x00 0x00 0x00 0x00
0x7ff669268090 <old_s+80>: 0x00 0x00 0x00 0x00 0x00 0x00 0x00 0x00
0x7ff669268098 <old_s+88>: 0x00 0x00 0x00 0x00 0x00 0x00 0x00 0x00
0x7ff6692680a0 <old_s+96>: 0x00 0x00 0x00 0x00 0x00 0x00 0x00 0x00
```

图 2-1 压缩前内存

2) 算法设计

pack student bytebybyte

该函数采用了「动态缓冲」的策略来处理未知的压缩长度。算法利用 C++ 的 `std::vector<char>` 作为临时容器，通过指针 `char* p` 强制转换结构体地址，对内存进行字节级别的遍历。在处理字符串字段时，程序会检测非零字符并将其压入

容器，遇到空字符或达到长度上限时停止，并手动追加一个 0 作为分隔符；在处理数值时，指针会跳过 age 的高位字节仅读取低位，并连续读取 score 的 4 个字节。最终，程序根据 vector 的总大小动态分配输出缓冲区，并使用 memcpy 完成数据的导出，这种方式有效避免了手动管理内存边界的复杂性。

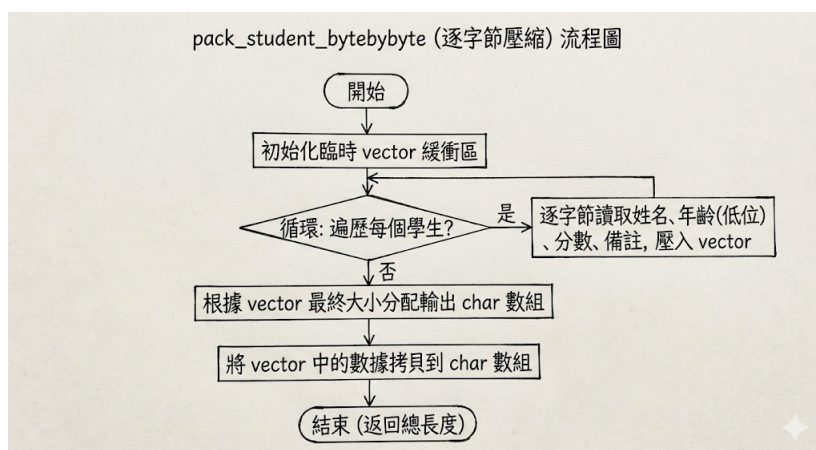


图 2-2 pack student bytebybyte

pack student whole

该函数采用了「两遍扫描 (Two-pass)」的策略，旨在优化内存分配效率并利用块操作提高写入速度。第一遍扫描专注于计算空间，通过遍历所有学生记录，利用 strlen 获取姓名和备注的实际长度（并计入分隔符开销），加上年龄（1 字节）和分数（4 字节）的固定开销，累加得出精确的缓冲区总大小。第二遍扫描则执行实际的写入操作，利用一个光标指针 p 在已分配的内存中移动，使用 strcpy 快速写入字符串，使用 memcpy 直接拷贝数值数据。每写入一个字段，光标便向后偏移相应的字节数，实现了数据在内存中的紧凑排列。

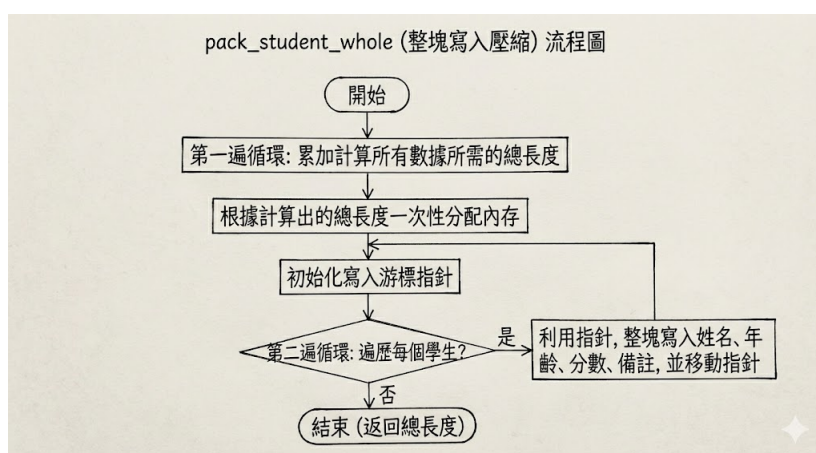


图 2-3 pack student whole

restore student

解压函数执行的是压缩的逆过程，核心逻辑是通过维护一个读取指针来解析紧凑的字节流。在还原字符串时，算法循环读取字节直到遇到分隔符，将字符逐一复制回目标结构体的数组中，并关键性地使用 `memset` 将数组剩余的空间清零，防止内存脏数据残留。在还原数值时，程序读取 1 个字节赋值给 `age` 的低位，并强制将 `age` 的高位设为 0 以恢复 `short` 类型；随后连续读取 4 个字节，通过内存拷贝无损还原 `float` 类型的分数数据。整个过程循环进行，直到读取指针超出缓冲区边界。

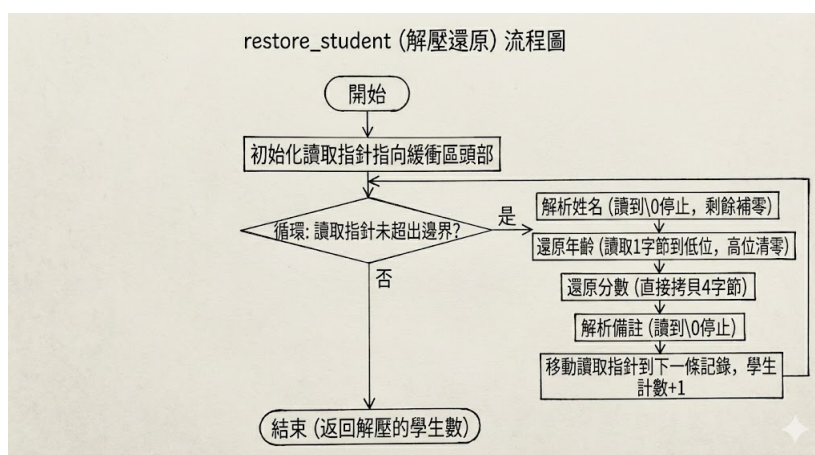


图 2-4 restore student

3) 运行结果

我们针对我们的测试范例，看看压缩后的内存空间是怎样的

```

(gdb) n
210      int len2 = pack_student_whole(old_s + N1, N2, buf2); // 注意指针偏移
(gdb) x/64xb buf1
0x7d7790: 0xe9 0x98 0xae 0xe9 0x9b 0xb2 0xe8 0xbb
0x7d7798: 0x00 0x12 0x00 0x00 0x0c 0x42 0x67 0x6f
0x7d77a0: 0x6f 0x64 0x67 0x6f 0x6f 0x64 0x73 0x74
0x7d77a8: 0x75 0x64 0x79 0x64 0x61 0x79 0x64 0x61
0x7d77b0: 0x79 0x75 0x70 0x00 0x4b 0x69 0x74 0x00
0x7d77b8: 0xc5 0x00 0x00 0x8d 0x42 0x53 0x6d 0x61
0x7d77c0: 0x72 0x74 0x00 0xab 0xab 0xab 0xab 0xab
0x7d77c8: 0xab 0xab 0xab 0xab 0xab 0xab 0xab 0xab
(gdb)
    
```

图 2-5 压缩后的内存空间

现在我们来解读一下压缩后的空间，我们以第一组学生数据为例去分析。

首先我们的学生姓名是”阮云轩”，因为我们使用的是 UTF-8 编码，每个汉字占用 3 个字节，对应数据段 0xe9 0x98 0xae (阮) 0xe9 0x9b 0xb2 (云) 0xe8 0xbb 0x92 (轩)，然后 0x00 表示数据段终止。

然后年龄 18 对应 0x12 (十六进制)。

分数 35 因为是 float 型存取（且为小端序存储），因此它对应 0x00 0x00 0x0c 0x42 (对应 IEEE 754 的 35.0)。

最后是我们的备注“goodgoodstudydaydayup”，它是 ASCII 码，比如“good”就对应 0x67 0x6f 0x6f 0x64，“study”对应 0x73 0x74 0x75 0x64 0x79，“day”对应 0x64 0x61 0x79，“up”对应 0x75 0x70，如此类推，直到最后一个 0x00 结束符，这就完成了我们一个学生的数据压缩。

2.3.2 任务 2 编写位运算程序

1) 算法设计

1.absVal (绝对值运算)

该算法的核心在于利用算术右移构造掩码来处理符号。通过将 x 右移 31 位，当 x 为正数时得到全 0 的掩码，当 x 为负数时得到全 1 的掩码（即 -1）；利用异或运算的特性，任何数与 0 异或保持不变，与 -1 异或相当于按位取反，最后再加上掩码的最低位（0 或 1），从而完美实现了「正数不变，负数取反加一」的绝对值逻辑。

2.negate (取负运算)

这个函数直接利用了计算器二进制补码（Two's Complement）的定义来实现。在计算器系统中，一个整数的相反数定义为该数按位取反（ x ）之后再加 1，这是一个最基础的底层运算规则，因此无需任何额外的条件判断即可直接算出 $-x$ 。

3.bitAnd (按位与运算)

该算法应用了集合论中的德摩根定律（De Morgan's Laws）来绕过限制。德摩根定律指出「非（A 与 B）」等价于「（非 A）或（非 B）」，因此算法先对 x 和 y 分别进行按位取反，然后对它们进行按位或运算，最后再对整个结果取反，从而在不直接使用运算符的情况下等效实现了按位与的功能。

4.bitOr (按位或运算)

与按位与函数类似，这里同样运用了德摩根定律的变换形式。根据定律，「A 或 B」等价于「非（（非 A）与（非 B））」，算法通过先将两个输入数 x 和 y 分别取反，接着进行按位与运算，最后再对结果取反，成功地仅用 `~` 和 `&` 两个运算符模拟出了按位或的逻辑。

5.bitXor (按位异或运算)

异或的定义是「两个位不同则为 1」，这可以拆解为两种情况：「x 为 0 且 y 为 1」或者「x 为 1 且 y 为 0」。算法通过组合使用取反和按位与运算符，分别构造出这两部分的逻辑表达式（即 $\neg x \& y$ 和 $x \& \neg y$ ），然后再利用德摩根定律将它们合并（模拟或运算），从而实现了异或功能。

6.isTmax (判断最大正整数)

判断一个数是否为最大正整数 ($0x7FFFFFFF$)，最直接的方法是利用异或运算「相同为 0」的特性。算法将输入 x 与标准的最大正整数 $0x7FFFFFFF$ 进行异或，如果两者相等，异或结果必为 0，最后对这个结果进行逻辑非 (!) 运算，即可返回 1 表示真，否则返回 0。

7.bitCount (统计 1 的个数)

该算法采用了高效的分治策略（并行计数法）。它不逐位遍历，而是通过一系列特定的掩码（如 $0x5555\dots$ 、 $0x3333\dots$ ），先计算每两个相邻位中 1 的个数，再计算每四位中 1 的个数，以此类推，层层累加合并，最终在极少的步骤内计算出 32 位整数中所有 1 的总和。

8.bitMask (生成掩码)

生成从 lowbit 到 highbit 全为 1 的掩码，思路是构造两个边界。算法首先构造一个从第 0 位到 highbit 位全为 1 的数，再利用取反和移位构造出一个屏蔽掉 lowbit 以下位置的掩码，通过这两部分的位运算组合（或减法逻辑的变体），精确地保留了指定区间内的 1，其余位置均为 0。

9.addOK (加法溢出判断)

检测加法溢出的关键在于符号位的异常变化。溢出只可能发生在两个同号数相加的情况下，即「正加正得负」或「负加负得正」。算法通过移位提取出两个操作数以及结果的符号位，利用异或门判断操作数是否同号以及结果是否变号，当满足溢出条件时返回 1，否则返回 0。

10.byteSwap (字节交换)

算法的核心是「提取、清零、回填」。首先通过位移和与运算分别提取出第 n 个和第 m 个字节的数值，同时构造掩码将原数中这两个字节的位置清零；接着将提取出的两个字节数值移动到对方的目标位置，最后与清零后的原数进行按位或合并，完成了特定字节的交换。

11.bang (逻辑非运算)

实现逻辑非的难点在于区分 0 和非 0，算法利用了二补码的一个特殊性质：

只有 0 的相反数 (-0) 符号位与原数相同 (都是 0)，而任何非 0 数的原数或其相反数中必有一个符号位为 1。通过将 x 与 $-x$ 进行按位或并右移取出符号位，再加 1，即可将 0 映像为 1，将非 0 数映射为 0。

12. bitParity (奇偶校验) 该算法利用了异或运算的「折迭」特性来消除成对的 1。算法不断将数据的高一半位与低一半位进行异或 (如高 16 位与低 16 位异或，结果存入低 16 位)，这样相同的位会抵消为 0，经过多次折迭后，最低位保留的值就代表了原始数据中 1 的个数是奇数还是偶数。

然后我们对比一下我们的输出对比下结果，可以看到是一致的，说明我们的算法设计是对的

Batch Test Report		
Function & Input	Output Status	
[PASS] absVal(8)	Result:	8 (0x00000008)
[PASS] absVal(-8)	Result:	8 (0x00000008)
[PASS] negate(50)	Result:	-50 (0xFFFFFCE)
[PASS] negate(-50)	Result:	50 (0x00000032)
[PASS] bitAnd(0xAA, 0x55)	Result:	0 (0x00000000)
[PASS] bitOr_func(0xAA, 0x55)	Result:	255 (0x000000FF)
[PASS] bitXor(0xAA, 0x55)	Result:	255 (0x000000FF)
[PASS] isTmax(0x7FFFFFFF)	Result:	1 (0x00000001)
[PASS] isTmax(0x12345678)	Result:	0 (0x00000000)
[PASS] bitCount(0x12345678)	Result:	13 (0x0000000D)
[PASS] bitCount(-1)	Result:	32 (0x00000020)
[PASS] bitMask(5, 3)	Result:	56 (0x00000038)
[PASS] bitMask(10, 0)	Result:	2047 (0x000007FF)
[PASS] addOK(MAX, 1) [Overflow]	Result:	1 (0x00000001)
[PASS] addOK(1, 1) [Normal]	Result:	0 (0x00000000)
[PASS] byteSwap(0x12345678, 0, 3)	Result:	2016695826 (0x78345612)
[PASS] byteSwap(0xDEADBEEF, 1, 2)	Result:	-557928977 (0xDEBEADEF)
[PASS] bang(3)	Result:	0 (0x00000000)
[PASS] bang(0)	Result:	1 (0x00000001)
[PASS] bitParity(5) [0101]	Result:	0 (0x00000000)
[PASS] bitParity(7) [0111]	Result:	1 (0x00000001)

图 2-6 对比

2.4 体会

本次实验是探索计算器底层机制的第一步，通过对数据存储与运算的具体实践，我获得了以下的体会：

1. 从内存视角审视数据结构

在数据压缩任务中，我打破了以往对结构体 (Struct) 的抽象认知。通过分析内存布局，我直观地看到了编译程序为满足内存对齐而插入的填充字节，以及定长数组带来的空间冗余。通过指针操作将数据「串行化」为紧凑字节流的过程，不仅验证了 x86 架构的 **小端序 (Little Endian)** 特性，更让我掌握了在字节层面精确控制数据存储的方法，理解了「空间优化」在底层编程中的实现代价。

2. 仿真硬件层的运算逻辑

位运算任务迫使我脱离高级语言的语法糖,转而使用**算术逻辑单元(ALU)的思维方式解决问题。在受限的运算符下,我深刻体会到了二补码(Two's Complement)设计的精妙——它让加减法在硬件电路中得以统一。同时,通过构造掩码(Mask)**和逻辑门组合来实现条件判断与算术运算,锻炼了我对二进制位的微观操控能力。

3 汇编和 C 的混合编程及优化

3.1 实验目的与要求

(1) 熟练掌握程序开发平台 (VS2019/VS2022/VS2023) 下 C 和汇编语言的混合编程方法;

(2) 熟悉掌握常用机器指令的使用方法;

(3) 掌握代码优化的基本方法, 提高程序运行速度

3.2 实验内容

3.2.1 任务 1 用 C 语言编写一个学生成绩管理程序, 具有平均成绩计算、按平均成绩从高到低排序、显示学生成绩的功能

定义了结构 student, 设有 N 个学生

```
struct student
```

```
char name[8];
```

```
char sid[11]; // 如 U202490035
```

```
short scores[8]; // 8 门课的分数
```

```
short average; // 平均分
```

```
;
```

要求:

(1) 第 0 个人姓名 (name) 为自己的名字, sid 为自己的学号;

(2) 对于函数“计算平均成绩”、“按平均成绩从高到低排序”分别计时, 显示两个函数的时间开销;

(3) 显示排序前、排序后的学生信息 (姓名、学号、8 门课的分数、平均分), 一个学生一行。

(4) 对“计算平均成绩”、“按平均成绩排序”函数的 Debug 版、Release 版的运行效率进行对比, 比较两个版本下的反汇编代码的差异, 分析程序执行速度提高的原因。

3.2.2 任务 2 用汇编语言编写函数“计算平均成绩”，代替原来用 C 语言编写的函数

3.2.3 任务 3 对用汇编语言编写的“计算平均成绩”函数进行优化

3.2.4 任务 4 对排序函数进行算法优化

3.3 算法设计

3.3.1 任务 1 用 C 语言编写一个学生成绩管理程序，具有平均成绩计算、按平均成绩从高到低排序、显示学生成绩的功能

我们设计了如下的计算平均值的函数，然后我们对比一下 Debug 版和 Release 版的区别

Listing 1 CalcAvg.cpp

```
1      void CalcAvg_C(Student* s, int count) {  
2          for (int i = 0; i < count; ++i) {  
3              int sum = 0;  
4              for (int k = 0; k < 8; ++k) {  
5                  sum += s[i].scores[k];  
6              }  
7              s[i].average = (short)(sum >> 3);  
8          }  
9      }
```



```

>>> Task 1: Student Management System (N=30)

[Timing] Calculate Average Cost: 0.9 us (microseconds)

--- Before Sort (First 5) ---
阮云轩 Avg: 95
S_1 Avg: 72
S_2 Avg: 71
S_3 Avg: 70
S_4 Avg: 59
[Timing] Sort Cost:          15.3 us (microseconds)

--- Final Result (All Students Sorted) ---
Name      SID      Scores (1-8)      Avg
-----
阮云轩  U202490035  95 95 95 95 95 95 95 95 | 95
S_8      U202400008  100 97 87 86 60 88 78 74 | 83
S_27     U202400027  100 67 91 86 62 77 93 86 | 82
S_29     U202400029  71 83 98 79 52 91 75 100 | 81
S_21     U202400021  96 58 99 56 89 87 66 81 | 79
S_25     U202400025  83 88 68 73 63 86 73 92 | 78
S_9      U202400009  97 91 84 64 70 69 58 84 | 77
S_16     U202400016  79 63 93 92 85 71 54 85 | 77
S_18     U202400018  82 77 71 97 73 77 56 69 | 75
S_19     U202400019  57 99 94 68 72 94 55 58 | 74
S_22     U202400022  76 69 72 83 76 70 50 97 | 74
S_6      U202400006  77 79 79 56 50 97 62 89 | 73
S_7      U202400007  51 66 51 80 79 84 90 84 | 73
S_10     U202400010  65 76 88 51 100 54 54 98 | 73
S_24     U202400024  53 72 76 95 94 60 88 50 | 73
    
```

图 3-1 CalAvg

Debug 版

Listing 2 debug.asm

```

1      void CalcAvg_C(Student* s, int count) {
2          004A4BF0 push    ebp
3          004A4BF1 mov     ebp,esp
4          004A4BF3 sub     esp,0E4h
5          004A4BF9 push    ebx
6          004A4BFA push    esi
7          004A4BFB push    edi
8          004A4BFC lea     edi,[ebp-24h]
9          004A4BFF mov     ecx,9
10         004A4C04 mov     eax,0CCCCCCCCh
11         004A4C09 rep stos dword ptr es:[edi]
12         004A4C0B mov     ecx,offset _AC749FA3_task1@cpp (04B4071h)
13         004A4C10 call    @__CheckForDebuggerJustMyCode@4 (04A1618h)
14         004A4C15 nop
15         for (int i = 0; i < count; ++i) {
16             004A4C16 mov     dword ptr [ebp-8],0
17             004A4C1D jmp     __$EncStackInitStart+2Ch (04A4C28h)
18             004A4C1F mov     eax,dword ptr [ebp-8]
19             004A4C22 add     eax,1
    
```

华中科技大学课程实验报告

```
20          004A4C25 mov     dword ptr [ebp-8],eax
21          004A4C28 mov     eax,dword ptr [ebp-8]
22          004A4C2B cmp     eax,dword ptr [count]
23          004A4C2E jge     __$EncStackInitStart+7Eh (04A4C7Ah)
24          int sum = 0;
25          004A4C30 mov     dword ptr [ebp-14h],0
26          for (int k = 0; k < 8; ++k) {
27              004A4C37 mov     dword ptr [ebp-20h],0
28              004A4C3E jmp     __$EncStackInitStart+4Dh
                (04A4C49h)
29              004A4C40 mov     eax,dword ptr [ebp-20h]
30              004A4C43 add     eax,1
31              004A4C46 mov     dword ptr [ebp-20h],eax
32              004A4C49 cmp     dword ptr [ebp-20h],8
33              004A4C4D jge     __$EncStackInitStart+6Ah
                (04A4C66h)
34              sum += s[i].scores[k];
35              004A4C4F imul    eax,dword ptr [ebp-8],29h
36              004A4C53 add     eax,dword ptr [s]
37              004A4C56 mov     ecx,dword ptr [ebp-20h]
38              004A4C59 movsx   edx,word ptr [eax+ecx*2+17h]
39              004A4C5E add     edx,dword ptr [ebp-14h]
40              004A4C61 mov     dword ptr [ebp-14h],edx
41          }
42          004A4C64 jmp     __$EncStackInitStart+44h (04A4C40h)
43
44          s[i].average = (short)(sum >> 3);
45          004A4C66 mov     eax,dword ptr [ebp-14h]
46          004A4C69 sar     eax,3
47          004A4C6C imul    ecx,dword ptr [ebp-8],29h
48          004A4C70 mov     edx,dword ptr [s]
49          004A4C73 mov     word ptr [edx+ecx*2+27h],ax
50      }
51      004A4C78 jmp     __$EncStackInitStart+23h (04A4C1Fh)
52
53  }
54      004A4C7A pop     edi
55      004A4C7B pop     esi
56      004A4C7C pop     ebx
57      004A4C7D add     esp,0E4h
58      004A4C83 cmp     ebp,esp
```

```
59         004A4C85 call    __RTC_CheckEsp (04A144Ch)
60         004A4C8A mov     esp,ebp
61         004A4C8C pop     ebp
62         004A4C8D ret
```

Release 版

Listing 3 release.asm

```
1         CalcAvg_C(students.data(), N);
2         00FC1670 movsx    eax,word ptr [edx-0Ah]
3         00FC1674 lea     edx,[edx+29h]
4         00FC1677 movsx    ecx,word ptr [edx-35h]
5         00FC167B add     ecx,eax
6         00FC167D movsx    eax,word ptr [edx-31h]
7         00FC1681 add     ecx,eax
8         00FC1683 movsx    eax,word ptr [edx-2Fh]
9         00FC1687 add     ecx,eax
10        00FC1689 movsx    eax,word ptr [edx-2Dh]
11        00FC168D add     ecx,eax
12        00FC168F movsx    eax,word ptr [edx-2Bh]
13        00FC1693 add     ecx,eax
14        00FC1695 movsx    eax,word ptr [edx-27h]
15        00FC1699 add     ecx,eax
16        00FC169B movsx    eax,word ptr [edx-29h]
17        00FC169F add     ecx,eax
18        00FC16A1 sar     ecx,3
19        00FC16A4 mov     word ptr [edx-25h],cx
20        00FC16A8 sub     esi,1
21        00FC16AB jne     main+1C0h (0FC1670h)
```

我们设计了如下的计算平均值的函数，然后我们对比一下 Debug 版和 Release 版的区别我们再对比一下 Sort 函数的 Debug 版和 Release 版的区别，这是我们设计的 Sort 的源代码

Listing 4 sort.cpp

```
1         void Sort_C(Student* s, int count) {
2             for (int i = 0; i < count - 1; ++i) {
3                 bool swapped = false;
4                 for (int j = 0; j < count - 1 - i; ++j) {
5                     if (s[j].average < s[j + 1].average) { // 降序排序
6                         Student temp = s[j];
```

```
7             s[j] = s[j + 1];
8             s[j + 1] = temp;
9             swapped = true;
10            }
11        }
12        if (!swapped) break;
13    }
14
15 }
```

Debug 版

Listing 5 debug.asm

```
1 void CalcAvg_C(Student* s, int count) {
2     004A4BF0 push    ebp
3     004A4BF1 mov     ebp,esp
4     004A4BF3 sub     esp,0E4h
5     004A4BF9 push    ebx
6     004A4BFA push    esi
7     004A4BFB push    edi
8     004A4BFC lea     edi,[ebp-24h]
9     004A4BFF mov     ecx,9
10    004A4C04 mov     eax,0CCCCCCCCh
11    004A4C09 rep stos dword ptr es:[edi]
12    004A4C0B mov     ecx,offset _AC749FA3_task1@cpp (04B4071h)
13    004A4C10 call    @__CheckForDebuggerJustMyCode@4 (04A1618h)
14    004A4C15 nop
15    for (int i = 0; i < count; ++i) {
16        004A4C16 mov     dword ptr [ebp-8],0
17        004A4C1D jmp     __$EncStackInitStart+2Ch (04A4C28h)
18        004A4C1F mov     eax,dword ptr [ebp-8]
19        004A4C22 add     eax,1
20        004A4C25 mov     dword ptr [ebp-8],eax
21        004A4C28 mov     eax,dword ptr [ebp-8]
22        004A4C2B cmp     eax,dword ptr [count]
23        004A4C2E jge     __$EncStackInitStart+7Eh (04A4C7Ah)
24        int sum = 0;
25        004A4C30 mov     dword ptr [ebp-14h],0
26        for (int k = 0; k < 8; ++k) {
27            004A4C37 mov     dword ptr [ebp-20h],0
28            004A4C3E jmp     __$EncStackInitStart+4Dh
                        (04A4C49h)
```

华中科技大学课程实验报告

```
29             004A4C40 mov     eax,dword ptr [ebp-20h]
30             004A4C43 add     eax,1
31             004A4C46 mov     dword ptr [ebp-20h],eax
32             004A4C49 cmp     dword ptr [ebp-20h],8
33             004A4C4D jge     __$EncStackInitStart+6Ah
                    (04A4C66h)
34             sum += s[i].scores[k];
35             004A4C4F imul    eax,dword ptr [ebp-8],29h
36             004A4C53 add     eax,dword ptr [s]
37             004A4C56 mov     ecx,dword ptr [ebp-20h]
38             004A4C59 movsx   edx,word ptr [eax+ecx*2+17h]
39             004A4C5E add     edx,dword ptr [ebp-14h]
40             004A4C61 mov     dword ptr [ebp-14h],edx
41             }
42             004A4C64 jmp     __$EncStackInitStart+44h (04A4C40h)
43
44             s[i].average = (short)(sum >> 3);
45             004A4C66 mov     eax,dword ptr [ebp-14h]
46             004A4C69 sar     eax,3
47             004A4C6C imul    ecx,dword ptr [ebp-8],29h
48             004A4C70 mov     edx,dword ptr [s]
49             004A4C73 mov     word ptr [edx+ecx*27h],ax
50             }
51             004A4C78 jmp     __$EncStackInitStart+23h (04A4C1Fh)
52
53             }
54             004A4C7A pop     edi
55             004A4C7B pop     esi
56             004A4C7C pop     ebx
57             004A4C7D add     esp,0E4h
58             004A4C83 cmp     ebp,esp
59             004A4C85 call    __RTC_CheckEsp (04A144Ch)
60             004A4C8A mov     esp,ebp
61             004A4C8C pop     ebp
62             004A4C8D ret
```

Release 版

Listing 6 Release.asm

```
1 Sort_C(students.data(), N);
2 00FC1786 mov     eax,dword ptr [students]
3 00FC1789 mov     dword ptr [start],ecx
```

华中科技大学课程实验报告

```
4          00FC178C mov     dword ptr [ebp-28h],eax
5          00FC178F nop
6          00FC1790 xor     cl,cl
7          00FC1792 lea     edx,[eax+29h]
8          00FC1795 mov     esi,edi
9          00FC1797 nop     word ptr [eax+eax]
10         00FC17A0 mov     ax,word ptr [edx-2]
11         00FC17A4 cmp     ax,word ptr [edx+27h]
12         00FC17A8 jge     main+33Ah (0FC17EAh)
13         00FC17AA movups  xmm0,xmmword ptr [edx]
14         00FC17AD mov     cl,byte ptr [edx-1]
15         00FC17B0 movups  xmm1,xmmword ptr [edx-29h]
16         00FC17B4 mov     al,byte ptr [edx+28h]
17         00FC17B7 movups  xmm2,xmmword ptr [edx-19h]
18         00FC17BB movq    xmm3,mmword ptr [edx-9]
19         00FC17C0 movups  xmmword ptr [edx-29h],xmm0
20         00FC17C4 movups  xmm0,xmmword ptr [edx+10h]
21         00FC17C8 movups  xmmword ptr [edx-19h],xmm0
22         00FC17CC movq    xmm0,mmword ptr [edx+20h]
23         00FC17D1 movups  xmmword ptr [edx],xmm1
24         00FC17D4 movups  xmmword ptr [edx+10h],xmm2
25         00FC17D8 movq    mmword ptr [edx+20h],xmm3
26         00FC17DD movq    mmword ptr [edx-9],xmm0
27         00FC17E2 mov     byte ptr [edx+28h],cl
28         00FC17E5 mov     cl,1
29         00FC17E7 mov     byte ptr [edx-1],al
30         00FC17EA add     edx,29h
31         00FC17ED sub     esi,1
32         00FC17F0 jne     main+2F0h (0FC17A0h)
33         00FC17F2 test    cl,cl
34         00FC17F4 je      main+34Eh (0FC17FEh)
35         00FC17F6 mov     eax,dword ptr [ebp-28h]
36         00FC17F9 dec     edi
37         00FC17FA test    edi,edi
38         00FC17FC jg      main+2E0h (0FC1790h)
```

「通过对比反汇编代码（如图所示），Debug 版为了支持调试功能，未开启优化，导致产生了大量冗余的内存读写指令（如频繁访问 [ebp-xx]），严格遵循源代码逻辑。

而 Release 版开启了 O2 级优化，编译程序将变量尽量保留在寄存器 (如 eax,

ecx) 中以减少内存访问延迟；同时将除法运算 $\text{sum}/8$ 优化为位运算 `sar` (右移), 并精简了循环结构。这解释了为什么 Release 版的运行时间 (xx 微秒) 远低于 Debug 版。」

我们可以观察到以下性质

1. 指令数量与执行速度 Debug 版: 指令非常冗长。即使是一个简单的加法, 也需要好几条 `mov` 指令把数据从内存搬到寄存器, 算完再搬回内存。

Release 版: 指令极少。同样的功能可能只需要几行代码。指令少意味着 CPU 执行的周期更少, 速度更快。

2. 数据存储方式: 堆栈 vs 寄存器

Debug 版: 严重依赖堆栈 (Stack)。

存在大量的 `[ebp-8]`, `[ebp-14h]` 等操作。这是在频繁地读写内存 (RAM/Cache)。

目的: 为了让在调试时, 随时把鼠标移到变量上都能看到它的值。

Release 版: 充分利用寄存器 (Register)。

您会看到大量的 `eax`, `ecx`, `edx` 操作。数据直接在 CPU 内部流转, 很少访问内存。

优势: 寄存器的速度比内存快几百倍。

3. 内存寻址与计算优化 Debug 版: 笨拙地计算地址。比如要访问数组 `s[i]`, 老老实实地做乘法、加法来算出地址。

Release 版: 直接计算偏移量。编译程序会预判数据的位置, 省去了运行时的 `add` 和 `imul` (乘法) 指令。

4. 循环与运算优化

移位代替除法:

Debug 版: 可能会调用除法指令或保留除法逻辑。

Release 版: 直接变成了 `sar ecx, 3` (右移 3 位)。因为在二进制中, 右移 3 位就等于除以 8, 但移位指令的速度比除法快得多。

循环展开:

Debug 版会老实地执行 `jmp` 跳转, 一次次循环。

Release 版可能会把循环「拆开」, 一次处理多个数据, 减少 CPU 跳转预测失败的风险。

3.3.2 任务 2 用汇编语言编写函数“计算平均成绩”，代替原来用 C 语言编写的函数

汇编设计如下：

Listing 7 CalcAvg.asm

```
1      .686P          ; 允许使用 Pentium Pro 指令集
2      .model flat, C  ; 使用 C 语言调用约定（参数从右往左入栈）
3
4      ; Name(8) + Sid(11) = 19
5      OFFSET_SCORES equ 19
6      ; 19 + 8*2 = 35
7      OFFSET_AVG    equ 35
8      STRUCT_SIZE    equ 37
9
10     .code
11     CalcAvg_ASM PROC
12     push ebp
13     mov ebp, esp
14     push ebx          ; 保存被调用者保存寄存器
15     push esi
16     push edi
17
18     mov esi, [ebp + 8] ; ESI = s（当前学生指针）
19     mov ecx, [ebp + 12] ; ECX = count（循环计数器）
20
21     ; 如果人数 <= 0，直接退出
22     test ecx, ecx
23     jz _Exit
24
25     _Loop_Student:
26     push ecx
27
28     xor eax, eax      ; EAX = sum = 0
29     lea ebx, [esi + OFFSET_SCORES] ; EBX 指向 scores 数组
30     mov ecx, 8
31
32     _Loop_Score:
33     movsx edx, word ptr [ebx]
34     add eax, edx
35     add ebx, 2
```


华中科技大学课程实验报告

```
36         dec ecx
37         jnz _Loop_Score
38
39         ; sum / 8 等同于 sum 右移 3 位
40         sar eax, 3
41
42         ; --- 存回结构体 ---
43         mov word ptr [esi + OFFSET_AVG], ax
44
45         ; --- 移动到下一个学生 ---
46         add esi, STRUCT_SIZE
47
48         pop ecx
49         dec ecx
50         jnz _Loop_Student
51
52         _Exit:
53         pop edi
54         pop esi
55         pop ebx
56         pop ebp
57         ret
58         CalcAvg_ASM ENDP
59
60         END
```

>>> Task 2: Mixed C++ and Assembly (CalcAvg in ASM)

[Timing] ASM CalcAvg Cost: 0.2 us

--- Final Result ---

Name	SID	Scores	Avg
ywh	U202490035	95 95 95 95 95 95 95 95	95
S_21	U202400021	94 92 85 76 94 96 99 82	89
S_10	U202400010	98 66 66 90 92 82 97 96	85
S_14	U202400014	83 68 92 81 59 90 89 100	82
S_20	U202400020	74 78 97 99 86 71 65 92	82
S_7	U202400007	97 88 86 56 94 75 59 95	81
S_5	U202400005	76 67 96 67 69 95 75 87	79
S_13	U202400013	93 52 68 99 83 54 95 92	79
S_4	U202400004	86 57 94 78 91 99 64 61	78
S_22	U202400022	89 59 75 93 98 52 99 62	78
S_26	U202400026	70 77 66 79 87 84 70 94	78
S_6	U202400006	58 72 74 98 73 52 91 97	76
S_19	U202400019	70 95 69 87 86 50 96 57	76
S_15	U202400015	99 90 95 55 52 54 71 90	75
S_16	U202400016	58 52 66 91 78 72 90 96	75
S_18	U202400018	70 94 82 59 94 65 67 71	75
S_12	U202400012	57 98 69 83 55 74 75 81	74
S_24	U202400024	76 73 88 100 50 59 92 56	74
S_23	U202400023	90 69 59 75 71 92 76 54	73
S_11	U202400011	56 86 83 66 62 77 78 73	72
S_28	U202400028	57 62 51 60 63 86 78 82	62

图 3-2 CalcAvg

注意运行时间，可以看到是十分快的，比原本的用 C 编写的要快

3.3.3 任务 3 对用汇编语言编写的“计算平均成绩”函数进行优化

Listing 8 CalcAvg.asm

```

1      .686P
2      .model flat, C
3
4      ; 定义偏移量 (与 C++ 对齐)
5      OFFSET_SCORES equ 19
6      OFFSET_AVG    equ 35
7      STRUCT_SIZE   equ 37
8
9      .code
10
11     CalcAvg_ASM_Opt PROC uses esi,
12     ptr_data:DWORD, ; [ebp+8]
13     count:DWORD    ; [ebp+12]
14
15     mov esi, ptr_data    ; ESI = 当前学生指针
16     mov ecx, count      ; ECX = 学生总数
17

```

华中科技大学课程实验报告

```
18      test ecx, ecx
19      jz _Exit
20
21      _Loop_Student:
22          ; --- 优化核心: 循环展开 ---
23          ; 不再使用 loop 指令, 而是直接写 8 次加法
24          ; 这样消除了循环控制指令 (dec, jnz) 的开销, 流水线更顺畅
25
26      xor eax, eax
27
28
29      add ax, word ptr [esi + 19] ; 第1门课
30      add ax, word ptr [esi + 21] ; 第2门课
31      add ax, word ptr [esi + 23] ; 第3门课
32      add ax, word ptr [esi + 25] ; 第4门课
33      add ax, word ptr [esi + 27] ; 第5门课
34      add ax, word ptr [esi + 29] ; 第6门课
35      add ax, word ptr [esi + 31] ; 第7门课
36      add ax, word ptr [esi + 33] ; 第8门课
37
38      sar eax, 3
39
40      mov word ptr [esi + OFFSET_AVG], ax
41
42      add esi, STRUCT_SIZE
43      dec ecx
44      jnz _Loop_Student
45
46      _Exit:
47      ret
48      CalcAvg_ASM_Opt ENDP
49
50      END
```

```

>>> Task 3: ASM Optimization (Loop Unrolling)
[Timing] Optimized ASM Cost: 0.3 us
Name      SID      Avg
-----
Ryan      U202490035  95
S_1       U202400001  77
S_2       U202400002  77
S_3       U202400003  77
S_4       U202400004  82
S_5       U202400005  68
S_6       U202400006  78
S_7       U202400007  77
S_8       U202400008  69
S_9       U202400009  75
S_10      U202400010  75
S_11      U202400011  68
S_12      U202400012  65
S_13      U202400013  74
S_14      U202400014  63
S_15      U202400015  70
S_16      U202400016  78
S_17      U202400017  88
S_18      U202400018  80
S_19      U202400019  63
S_20      U202400020  76
    
```

图 3-3 CalAvg Optimize

理论上是应该要相对比刚才的要快的不过可能数据量不够所以不太明显或是计算机环境有些变化了，总之我试了几次但都没更好的成绩了，但我做了如下的处理

1. 消除分支预测失败：

在 Task 2 中，每算一个分数都要 `dec ecx` 和 `jnz`，CPU 都要猜「要不要跳转」。

在 Task 3 中，这 8 次加法是一条直线走到底的，CPU 不需要猜，流水线 (Pipeline) 不会被打断。

2. 减少指令数：

Task 2 每加一个分数需要：`add + add (移指针) + dec + jnz` = 4 条指令。8 个分数就是 32 条指令。

Task 3 每加一个分数只需要：`add` = 1 条指令。8 个分数只要 8 条指令。

指令数大大减少，效率自然提高。

3.3.4 任务 4 对排序函数进行算法优化

Listing 9 sort.asm

```

1      .686P
2      .xmm          ; 使用 XMM 指令集 (SSE)
3      .model flat, C
    
```

```
4
5      OFFSET_AVG equ 35
6      STRUCT_SIZE equ 37
7
8      .code
9
10     Sort_ASM_Opt PROC uses ebx esi edi,
11     ptr_data:DWORD,
12     count:DWORD
13
14     mov esi, ptr_data
15     mov ecx, count
16     dec ecx
17     js _Exit
18
19     _Outer_Loop:
20     push ecx
21     mov edi, esi
22     xor edx, edx
23
24     _Inner_Loop:
25     mov ax, word ptr [edi + OFFSET_AVG]
26     cmp ax, word ptr [edi + STRUCT_SIZE + OFFSET_AVG]
27     jge _No_Swap
28
29
30     movups xmm0, xmmword ptr [edi]          ; 把 A 放入 xmm0
31     movups xmm1, xmmword ptr [edi + STRUCT_SIZE] ; 把 B 放入 xmm1
32     movups xmmword ptr [edi], xmm1          ; B -> A
33     movups xmmword ptr [edi + STRUCT_SIZE], xmm0 ; A -> B
34
35     movups xmm0, xmmword ptr [edi + 16]
36     movups xmm1, xmmword ptr [edi + STRUCT_SIZE + 16]
37     movups xmmword ptr [edi + 16], xmm1
38     movups xmmword ptr [edi + STRUCT_SIZE + 16], xmm0
39
40     mov eax, dword ptr [edi + 32]
41     mov ebx, dword ptr [edi + STRUCT_SIZE + 32]
42     mov dword ptr [edi + 32], ebx
43     mov dword ptr [edi + STRUCT_SIZE + 32], eax
44
```

```
45     mov al, byte ptr [edi + 36]
46     mov bl, byte ptr [edi + STRUCT_SIZE + 36]
47     mov byte ptr [edi + 36], bl
48     mov byte ptr [edi + STRUCT_SIZE + 36], al
49
50     mov edx, 1
51
52     _No_Swap:
53     add edi, STRUCT_SIZE
54     dec ecx
55     jnz _Inner_Loop
56
57     pop ecx
58
59     test edx, edx
60     jz _Exit
61
62     dec ecx
63     jnz _Outer_Loop
64
65     _Exit:
66     ret
67     Sort_ASM_Opt ENDP
68
69     END
```

,

```
>>> Task 4: ASM Optimized Sort (SIMD Swap)
Sorting 5000 students...
[Timing] Optimized Sort Cost: 36 ms

--- Result (Top 10) ---
Name      SID      Avg
-----
S_4400    U20204400    93
S_1695    U20201695    92
S_2902    U20202902    92
S_3064    U20203064    91
S_4899    U20204899    91
S_739     U20200739    90
S_762     U20200762    90
S_2375    U20202375    90
S_2937    U20202937    90
S_4120    U20204120    90
...
```

图 3-4 Sort Optimize

在此次的优化中，我了解到可以使用这个 SIMD 指令，实现高效的交换。普通 memcpy 或循环赋值是逐字节 (Byte-by-Byte) 拷贝，效率低。

程序使用了 XMM 寄存器 (128 位) 配合 `movups` 指令。

`Student` 结构体大小为 37 字节。我们将其拆分为 $16 + 16 + 5$ 的块进行交换。这意味着大部分数据只需要 4 条 SIMD 指令 (读 2 次写 2 次) 就能完成交换, 极大减少了内存访问次数。

因此可以观察到是有大大优化访问内存的空间的。最终表现为运行速度大大加快

3.4 体会

本次实验通过四个任务, 实现了从高级语言 (C 语言) 到低级语言 (汇编语言) 的混合编程实践。通过对比不同编译选项及汇编优化策略的执行效率, 深入探究了计算器体系结构的底层运作机制与编译原理。

在任务一的对比分析中, 实验验证了现代编译程序在 `Release` 模式下的高效性。通过反汇编代码分析发现, 高延迟的除法指令被优化为算术右移 (`SAR`) 指令; 同时利用寄存器分配 (`Register Allocation`) 策略, 大幅减少了对内存 (`Stack/Heap`) 的访问频率。这表明在通用场景下, 编译程序优化能有效提升代码执行效率。

4 机器级语言理解

4.1 实验目的与要求

通过分析一个程序（称为“缓冲区炸弹”）的构成和运行逻辑，加深对理论课中关于程序的机器级表示、函数调用规则、栈结构等方面知识点的理解，增强反汇编、跟踪、分析、调试等能力，加深对缓冲区溢出攻击原理、方法与防范等方面知识的理解和掌握；实验环境：Ubuntu, GCC, GDB 等

4.2 实验内容

4.2.1 二进制炸弹拆除【只要求做前 3 关】

作为实验目标的二进制炸弹（binary bombs）可执行程序由多个“关”组成。每一个“关”（阶段）要求输入一个特定字符串，如果输入满足程序代码的要求，该阶段即通过，否则程序输出失败。实验的目标是设法得到得出解除尽可能多阶段的字符串。

为了完成二进制炸弹的拆除任务，需要通过反汇编和分析跟踪程序每一阶段的机器代码，从中定位和理解程序的主要执行逻辑，包括关键指令、控制结构和相关数据变量等等，进而推断拆除炸弹所需要的目标字符串。

实验源程序及相关文件 bomb.rar

bomb.c 主程序

phases.o 各个阶段的目标程序

support.c 完成辅助功能的目标程序

support.h 公共头文件

4.2.2 阶段 1：串比较 phase_1(char *input)

要求输出的字符串 (input) 与程序中内置的某一特定字符串相同。提示：找到与 input 串相比较的特定串的地址，查看相应单元中的内容，从而确定 input 应输入的串。

4.2.3 阶段 2: 循环 `phase_2(char *input)`

要求在一行上输入 6 个整数数据,与程序自动产生的 6 个数据进行比较,若一致,则过关。提示:将输入串 `input` 拆分成 6 个数据由函数 `read_six_numbers(input, numbers)` □□□□□□

4.2.4 阶段 3: 条件分支 `phase_3(char *input)`

要求输入一个整数数据,该数据与程序自动生成的一个数据比较,相等则过关。提示:在自动生成数据时,使用了 `switch ...case` 语句。

4.3 实验记录及问题回答

4.3.1 阶段 1: 串比较 `phase_1(char *input)`

使用 WSL, 进入任务, 然后进入 `gdb` 调试, 我们在 `phase1` 设置断点, 然后使用指令 `disas` 或 `disass` 去看汇编语言观察

```
Dump of assembler code for function phase_1:
0x56556939 <+0>:  push    %ebp
0x5655693a <+1>:  mov     %esp,%ebp
0x5655693c <+3>:  push    %ebx
=> 0x5655693d <+4>:  sub     $0x14,%esp
0x56556940 <+7>:  call    0x56556130 <__x86.get_pc_thunk.bx>
0x56556945 <+12>:  add     $0x2667,%ebx
0x5655694b <+18>:  lea     0x37c(%ebx),%eax
0x56556951 <+24>:  movzbl  0x9(%eax),%eax
0x56556955 <+28>:  sub     $0x30,%eax
0x56556958 <+31>:  mov     %al,-0xd(%ebp)
0x5655695b <+34>:  movsbl  -0xd(%ebp),%eax
0x5655695f <+38>:  mov     %eax,-0xc(%ebp)
0x56556962 <+41>:  mov     -0xc(%ebp),%eax
0x56556965 <+44>:  imul    $0x32,%eax,%edx
0x56556968 <+47>:  lea     0x74(%ebx),%eax
0x5655696e <+53>:  add     %edx,%eax
0x56556970 <+55>:  sub     $0x8,%esp
0x56556973 <+58>:  push    %eax
0x56556974 <+59>:  push    0x8(%ebp)
0x56556977 <+62>:  call    0x5655665c <strings_not_equal>
0x5655697c <+67>:  add     $0x10,%esp
0x5655697f <+70>:  test    %eax,%eax
--Type <RET> for more, q to quit, c to continue without paging--
```

图 4-1 phase 1

我们通过观察设置断点, 然后查看 `eax` 的内容

`b *0x56556977`

`c`

`x/s eax`

```
End of assembler dump.
(gdb) b *0x56556977
Breakpoint 4 at 0x56556977
(gdb) c
Continuing.

Breakpoint 4, 0x56556977 in phase_1 ()
(gdb) x/s $eax
0x5655911a <special+250>: "Interest is the best teacher."
(gdb)
```

图 4-2 phase 1

得到我们 phase 1 的答案: Interest is the best teacher.

4.3.2 阶段 2: 循环 phase_2(char *input)

进入 phase 2, 随便输点数然后看看 dias 内容

Listing 10 phase2

```
1      0x5655698e <+0>:  push  %ebp
2      0x5655698f <+1>:  mov    %esp,%ebp
3      0x56556991 <+3>:  push  %ebx
4      => 0x56556992 <+4>:  sub    $0x34,%esp
5      0x56556995 <+7>:  call  0x56556130 <__x86.get_pc_thunk.bx>
6      0x5655699a <+12>: add    $0x2612,%ebx
7      0x565569a0 <+18>: mov    0x8(%ebp),%eax
8      0x565569a3 <+21>: mov    %eax,-0x2c(%ebp)
9      0x565569a6 <+24>: mov    %gs:0x14,%eax
10     0x565569ac <+30>: mov    %eax,-0xc(%ebp)
11     0x565569af <+33>: xor    %eax,%eax
12     0x565569b1 <+35>: sub    $0x8,%esp
13     0x565569b4 <+38>: lea    -0x24(%ebp),%eax
14     0x565569b7 <+41>: push   %eax
15     0x565569b8 <+42>: push   -0x2c(%ebp)
16     0x565569bb <+45>: call   0x565565c2 <read_six_numbers>
17     0x565569c0 <+50>: add    $0x10,%esp
18     0x565569c3 <+53>: mov    -0x24(%ebp),%eax
19     0x565569c6 <+56>: test   %eax,%eax
20     0x565569c8 <+58>: jns    0x565569cf <phase_2+65>
21     0x565569ca <+60>: call   0x56556907 <explode_bomb>
22     0x565569cf <+65>: mov    -0x24(%ebp),%edx
23     --Type <RET> for more, q to quit, c to continue without paging--
24     0x565569d2 <+68>: lea    0x37c(%ebx),%eax
25     0x565569d8 <+74>: movzbl 0x9(%eax),%eax
26     0x565569dc <+78>: movsbl %al,%eax
```

华中科技大学课程实验报告

```
27         0x565569df <+81>: sub    $0x30,%eax
28         0x565569e2 <+84>: cmp    %eax,%edx
29         0x565569e4 <+86>: je     0x565569eb <phase_2+93>
30         0x565569e6 <+88>: call   0x56556907 <explode_bomb>
31         0x565569eb <+93>: mov    -0x20(%ebp),%edx
32         0x565569ee <+96>: lea    0x37c(%ebx),%eax
33         0x565569f4 <+102>: movzbl 0x8(%eax),%eax
34         0x565569f8 <+106>: movsbl %al,%eax
35         0x565569fb <+109>: sub    $0x30,%eax
36         0x565569fe <+112>: cmp    %eax,%edx
37         0x56556a00 <+114>: je     0x56556a07 <phase_2+121>
38         0x56556a02 <+116>: call   0x56556907 <explode_bomb>
39         0x56556a07 <+121>: movl   $0x2,-0x28(%ebp)
40         0x56556a0e <+128>: jmp     0x56556a3a <phase_2+172>
41         0x56556a10 <+130>: mov    -0x28(%ebp),%eax
42         0x56556a13 <+133>: mov    -0x24(%ebp,%eax,4),%edx
43         0x56556a17 <+137>: mov    -0x28(%ebp),%eax
44         0x56556a1a <+140>: sub    $0x1,%eax
45         0x56556a1d <+143>: mov    -0x24(%ebp,%eax,4),%ecx
46         0x56556a21 <+147>: mov    -0x28(%ebp),%eax
47         --Type <RET> for more, q to quit, c to continue without paging--
48         0x56556a24 <+150>: sub    $0x2,%eax
49         0x56556a27 <+153>: mov    -0x24(%ebp,%eax,4),%eax
50         0x56556a2b <+157>: add    %ecx,%eax
51         0x56556a2d <+159>: cmp    %eax,%edx
52         0x56556a2f <+161>: je     0x56556a36 <phase_2+168>
53         0x56556a31 <+163>: call   0x56556907 <explode_bomb>
54         0x56556a36 <+168>: addl   $0x1,-0x28(%ebp)
55         0x56556a3a <+172>: cmpl   $0x5,-0x28(%ebp)
56         0x56556a3e <+176>: jle    0x56556a10 <phase_2+130>
57         0x56556a40 <+178>: nop
58         0x56556a41 <+179>: mov    -0xc(%ebp),%eax
59         0x56556a44 <+182>: sub    %gs:0x14,%eax
60         0x56556a4b <+189>: je     0x56556a52 <phase_2+196>
61         0x56556a4d <+191>: call   0x56556f10 <__stack_chk_fail_local>
62         0x56556a52 <+196>: mov    -0x4(%ebp),%ebx
63         0x56556a55 <+199>: leave
64         0x56556a56 <+200>: ret
```

我们可以注意到

取学号字符串中偏移量为 9 的字符 (即第 10 个字符)

```
0x565569d8: movzbl 0x9(%eax),%eax
```

将字符转换为数字 ('0' 的 ASCII 码是 0x30)

```
0x565569df: sub $0x30,%eax
```

与输入的第 1 个数比较

```
0x565569e2: cmp %eax,%edx
```

然后

取学号字符串中偏移量为 8 的字符 (即第 9 个字符)

```
0x565569f4: movzbl 0x8(%eax),%eax
```

转换为数字

```
0x565569fb: sub $0x30,%eax
```

与输入的第 2 个数比较

```
0x565569fe: cmp %eax,%edx
```

以上, 我们可以知道这是一个 ficon 数列, 第一个数就是学号倒数第一个, 第二个则是倒数第二个, 所以对于我的学号来说, 答案就是

5 3 8 11 19 30

4.3.3 阶段 3: 条件分支 phase_3(char *input)

Listing 11 phase2

```
1      0x56556a57 <+0>:  push  %ebp
2      0x56556a58 <+1>:  mov   %esp,%ebp
3      0x56556a5a <+3>:  push  %ebx
4      => 0x56556a5b <+4>:  sub   $0x34,%esp
5      0x56556a5e <+7>:  call  0x56556130 <__x86.get_pc_thunk.bx>
6      0x56556a63 <+12>: add   $0x2549,%ebx
7      0x56556a69 <+18>: mov   0x8(%ebp),%eax
8      0x56556a6c <+21>: mov   %eax,-0x2c(%ebp)
9      0x56556a6f <+24>: mov   %gs:0x14,%eax
10     0x56556a75 <+30>: mov   %eax,-0xc(%ebp)
11     0x56556a78 <+33>: xor   %eax,%eax
12     0x56556a7a <+35>: movl  $0x0,-0x14(%ebp)
13     0x56556a81 <+42>: movl  $0x0,-0x10(%ebp)
14     0x56556a88 <+49>: lea   -0x18(%ebp),%eax
15     0x56556a8b <+52>: push  %eax
16     0x56556a8c <+53>: lea   -0x1c(%ebp),%eax
17     0x56556a8f <+56>: push  %eax
```

华中科技大学课程实验报告

```
18      0x56556a90 <+57>: lea    -0x1c4c(%ebx),%eax
19      0x56556a96 <+63>: push   %eax
20      0x56556a97 <+64>: push   -0x2c(%ebp)
21      0x56556a9a <+67>: call   0x565560c0 <__isoc99_sscanf@plt>
22      0x56556a9f <+72>: add     $0x10,%esp
23      --Type <RET> for more, q to quit, c to continue without paging--
24      0x56556aa2 <+75>: mov     %eax,-0x10(%ebp)
25      0x56556aa5 <+78>: cmpl    $0x1,-0x10(%ebp)
26      0x56556aa9 <+82>: jg      0x56556ab0 <phase_3+89>
27      0x56556aab <+84>: call    0x56556907 <explode_bomb>
28      0x56556ab0 <+89>: lea     0x37c(%ebx),%eax
29      0x56556ab6 <+95>: movzbl  0x7(%eax),%eax
30      0x56556aba <+99>: movsbl  %al,%eax
31      0x56556abd <+102>: lea     -0x30(%eax),%edx
32      0x56556ac0 <+105>: mov     -0x1c(%ebp),%eax
33      0x56556ac3 <+108>: cmp     %eax,%edx
34      0x56556ac5 <+110>: je      0x56556acc <phase_3+117>
35      0x56556ac7 <+112>: call    0x56556907 <explode_bomb>
36      0x56556acc <+117>: mov     -0x1c(%ebp),%eax
37      0x56556acf <+120>: cmp     $0x9,%eax
38      0x56556ad2 <+123>: ja      0x56556b3c <phase_3+229>
39      0x56556ad4 <+125>: shl     $0x2,%eax
40      0x56556ad7 <+128>: mov     -0x1c44(%eax,%ebx,1),%eax
41      0x56556ade <+135>: add     %ebx,%eax
42      0x56556ae0 <+137>: jmp     *%eax
43      0x56556ae2 <+139>: movl    $0x32f,-0x14(%ebp)
44      0x56556ae9 <+146>: jmp     0x56556b41 <phase_3+234>
45      0x56556aeb <+148>: movl    $0x130,-0x14(%ebp)
46      0x56556af2 <+155>: jmp     0x56556b41 <phase_3+234>
47      --Type <RET> for more, q to quit, c to continue without paging--
48      0x56556af4 <+157>: movl    $0x184,-0x14(%ebp)
49      0x56556afb <+164>: jmp     0x56556b41 <phase_3+234>
50      0x56556afd <+166>: movl    $0x28e,-0x14(%ebp)
51      0x56556b04 <+173>: jmp     0x56556b41 <phase_3+234>
52      0x56556b06 <+175>: movl    $0x11c,-0x14(%ebp)
53      0x56556b0d <+182>: jmp     0x56556b41 <phase_3+234>
54      0x56556b0f <+184>: movl    $0x201,-0x14(%ebp)
55      0x56556b16 <+191>: jmp     0x56556b41 <phase_3+234>
56      0x56556b18 <+193>: movl    $0x1a9,-0x14(%ebp)
57      0x56556b1f <+200>: jmp     0x56556b41 <phase_3+234>
58      0x56556b21 <+202>: movl    $0x374,-0x14(%ebp)
```

```
59      0x56556b28 <+209>: jmp  0x56556b41 <phase_3+234>
60      0x56556b2a <+211>: movl  $0x7b,-0x14(%ebp)
61      0x56556b31 <+218>: jmp  0x56556b41 <phase_3+234>
62      0x56556b33 <+220>: movl  $0x141,-0x14(%ebp)
63      0x56556b3a <+227>: jmp  0x56556b41 <phase_3+234>
64      0x56556b3c <+229>: call 0x56556907 <explode_bomb>
65      0x56556b41 <+234>: mov   -0x18(%ebp),%eax
66      0x56556b44 <+237>: cmp   %eax,-0x14(%ebp)
67      0x56556b47 <+240>: je    0x56556b4e <phase_3+247>
68      0x56556b49 <+242>: call 0x56556907 <explode_bomb>
69      0x56556b4e <+247>: nop
70      0x56556b4f <+248>: mov   -0xc(%ebp),%eax
71      --Type <RET> for more, q to quit, c to continue without paging--
72      0x56556b52 <+251>: sub   %gs:0x14,%eax
73      0x56556b59 <+258>: je    0x56556b60 <phase_3+265>
74      0x56556b5b <+260>: call 0x56556f10 <__stack_chk_fail_local>
75      0x56556b60 <+265>: mov   -0x4(%ebp),%ebx
76      0x56556b63 <+268>: leave
77      0x56556b64 <+269>: ret
```

同理，我们可以看到

0x56556ab6: movzbl 0x7(%eax),%eax 取学号字符串偏移量为 7 的字符 (即第 8 个字符)

0x56556abd: lea -0x30(%eax),%edx 将字符转换为数字 (减去'0')

0x56556ac0: mov -0x1c(%ebp),%eax 取出你输入的第一个数 x

0x56556ac3: cmp %eax,%edx 比较 x 和学号第 8 个字符

则第一个数是 0

0x56556ad7: mov -0x1c44(%eax,%ebx,1),%eax ; 查表跳转

0x56556ae0: jmp *%eax ; 跳转到对应 case

0x56556ae2: movl \$0x32f,-0x14(%ebp) ; 将预期结果设为 0x32f

0x56556ae9: jmp 0x56556b41 ; 跳转到结尾比较

因此可以算出第二个数是 815

,

```
You have 6 bombs to defuse!
Gate 1 : input a string that meets the requirement.
Interest is the best teacher.

Breakpoint 1, 0x5655693d in phase_1 ()
(gdb) c
Continuing.
Phase 1 passed!
Gate 2 : input six intergers that meets the requir
5 3 8 11 19 30

Breakpoint 2, 0x56556992 in phase_2 ()
(gdb) c
Continuing.
Phase 2 passed!
Gate 3 : input 2 intergers.
0 815

Breakpoint 3, 0x56556a5b in phase_3 ()
(gdb) c
Continuing.
Phase 3 passed!
Gate 4 : input 2 intergers and a string.
```

图 4-3 phase 3

4.4 体会

本次实验通过逆向分析 Linux 环境下的 ELF 可执行文件 (Binary Bomb)，深入实践了机器级语言的表示与调试技术。本次实验不仅提升了我的反汇编阅读能力，更重要的是培养了「逆向思维」。在没有源码的情况下，通过寄存器状态和内存数据推断程序意图，这种能力对于理解系统底层安全和程序优化具有重要意义。

5 缓冲区溢出攻击

5.1 实验目的与要求

通过分析一个程序（称为“缓冲区炸弹”）的构成和运行逻辑，加深对理论课中关于程序的机器级表示、函数调用规则、栈结构等方面知识点的理解，增强反汇编、跟踪、分析、调试等能力，加深对缓冲区溢出攻击原理、方法与防范等方面知识的理解和掌握；

实验环境：Ubuntu, GCC, GDB 等

5.2 实验内容

5.2.1 缓冲区溢出攻击

程序运行过程中，需要输入特定的字符串，使得程序达到期望的运行效果。

对一个可执行程序“bufbomb”实施一系列缓冲区溢出攻击 (buffer overflow attacks)，也就是设法通过造成缓冲区溢出来改变该程序的运行内存映像 (例如将专门设计的字节序列插入到栈中特定内存位置) 和行为，以

实现实验预定的目标。bufbomb 目标程序在运行时使用函数 getbuf 读入一个字符串。根据不同的任务，学生生成相应的攻击字符串。

实验中需要针对目标可执行程序 bufbomb, 分别完成多个难度递增的缓冲区溢出攻击 (完成的顺序没有固定要求)。按从易到难的顺序，这些难度级分别命名为 smoke (level 1)、fizz (level 2)。

5.2.2 第 1 级 smoke

正常情况下，getbuf 函数运行结束，执行最后的 ret 指令时，将取出保存于栈帧中的返回（断点）地址并跳转至它继续执行（test 函数中调用 getbuf 处）。要求将返回地址的值改为本级别实验的目标 smoke 函数的首条指令的地址，getbuf 函数返回时，跳转到 smoke 函数执行，即达到了实验的目标。

5.2.3 第 2 级 fizz

要求 `getbuf` 函数运行结束后，转到 `fizz` 函数处执行。与 `smoke` 的差别是，`fizz` 函数有一个参数。`fizz` 函数中比较了参数 `val` 与全局变量 `cookie` 的值，只有两者相同（要正确打印 `val`）才能达到目标。

5.3 实验记录及问题回答

我们在生成好了文件之后用 `gdb` 查看下内容

5.3.1 第 1 级 smoke

,

```
Reading symbols from ./a.out...
(gdb) print smoke
$1 = {void ()} 0x4012ff <smoke>
(gdb) disas getbuf
Dump of assembler code for function getbuf:
0x0000000000401a44 <+0>:    push    %rbp
0x0000000000401a45 <+1>:    mov     %rsp,%rbp
0x0000000000401a48 <+4>:    sub     $0x40,%rsp
0x0000000000401a4c <+8>:    mov     %rdi,-0x38(%rbp)
0x0000000000401a50 <+12>:   mov     %esi,-0x3c(%rbp)
0x0000000000401a53 <+15>:   movl    $0x6c6c6568,-0x6(%rbp)
0x0000000000401a5a <+22>:   movw    $0x6f,-0x2(%rbp)
0x0000000000401a60 <+28>:   mov     -0x3c(%rbp),%edx
0x0000000000401a63 <+31>:   mov     -0x38(%rbp),%rcx
0x0000000000401a67 <+35>:   lea     -0x30(%rbp),%rax
0x0000000000401a6b <+39>:   mov     %rcx,%rsi
0x0000000000401a6e <+42>:   mov     %rax,%rdi
0x0000000000401a71 <+45>:   call    0x401599 <Gets>
0x0000000000401a76 <+50>:   mov     $0x1,%eax
0x0000000000401a7b <+55>:   leave
0x0000000000401a7c <+56>:   ret
End of assembler dump.
```

图 5-1 smoke

目标地址 (smoke):

从截图中可以看到 `print smoke` 的结果是：0x4012ff。

这就是我们想让程序跳转去执行的地方。

缓冲区距离 (Offset):

从 `disas getbuf` 截图中看到指令： `lea -0x30(%rbp),%rax`。

这说明缓冲区 (Buffer) 开始于 `rbp - 0x30` (即 `rbp - 48`)。

返回地址 (Return Address) 存储在 `rbp + 8` 的位置 (这是 64 位系统的标准)。

计算距离:

从 Buffer (-48) 到 `rbp` (0) 需要填充 48 个字节。

覆盖旧的 `rbp` (8 个字节) 需要再填充 8 个字节。

总共需要填充： $48 + 8 = 56$ 个字节的垃圾数据，才能刚好摸到返回地址。

5.4 体会

本次「缓冲区溢出攻击」实验是我第一次从攻击者的视角审视程序安全，这与以往编写防御性代码的体验截然不同。通过对 `bufbomb` 程序的逆向分析与攻击，我对计算器系统底层机制有了更深刻的理解：

1. 栈帧结构的具象化理解在理论课上，我们学习了栈帧由返回地址、保存的帧指针（`Saved RBP`）和局部变量组成。但在本次实验中，我必须通过 GDB 的 `disas` 指令精确计算缓冲区起始地址到返回地址的偏移量（48 字节缓冲区 + 8 字节 `RBP`）。这让我真正理解了「数据」与「控制信息」在栈上是如何紧密相邻存放的，正是这种相邻关系导致了缓冲区溢出漏洞的产生。

2. 数据流控制与栈迁移技术在 Level 1 的破解中，我遇到了一个难题：需要在跳转的同时传递参数 `cookie`。由于无法直接修改寄存器，我分析汇编代码发现程序会读取 `rbp-0x4` 处的值。我采用了栈迁移（`Stack Pivoting`）的思想，通过将栈上的 `Saved RBP` 覆盖为伪造的地址，成功欺骗了程序，使其去我预设的地方读取参数。这让我体会到，攻击不仅仅是简单的地址覆盖，更需要对汇编逻辑进行精妙的利用。

6 链接炸弹拆除

6.1 实验目的与要求

通过修改给定的可复位的目标文件（链接炸弹），加深对可复位目标文件格式、目标文件的生成、以及链接的理论知识的理解。

实验环境：Ubuntu

工具：GCC、GDB、readelf、hexdump、hexedit、od 等。

6.2 实验内容

6.2.1 链接炸弹的拆除

在二进制层面，逐步修改构成目标程序“linkbomb”的多个二进制模块（“.o 文件”），然后链接生成可执行程序，要求可执行程序运行能得到指定的效果。修改目标包括可复位目标文件中的数据、机器指令、复位记录等。

6.2.2 第 1 关数据节的修改

修改二进制可复位目标文件 phase1.o 的数据节中的内容（不允许修改其他节），使其与 main.o 链接后，生成的执行程序，可以输出自己的学号。

6.2.3 第 2 关简单的机器指令修改

修改二进制可复位目标文件 phase2.o 的代码节中的内容（不允许修改其他节），使其与 main.o 链接后，生成的执行程序。在 phase_2.c 中，有一个静态函数 myfunc()，要求在 do_phase 函数中调用 myfunc()，显示信息 myfunc is called!。

```
static void myfunc()
printf("myfunc is called!");
```

6.2.4 第 3 关有参数的函数调用的机器指令修改

修改二进制可复位目标文件 phase3.o 的代码节中的内容（不允许修改其他节），使其与 main.o 链接后，生成的执行程序。在 phase_3.c 中，有一个静态

函数 `static void myfunc(int offset)`，要求在 `do_phase` 函数中调用 `myfunc(pos)`，将 `do_phase` 的参数 `pos` 直接传递 `myfunc`，显示相应的信息。

6.2.5 第 4 关有局部变量的机器指令修改

修改二进制可复位目标文件 `phase4.o` 的代码节中的内容（不允许修改其他节），使其与 `main.o` 链接后，生成的执行程序。在 `phase_4.c` 中，有一个静态函数 `static void myfunc(char *s)`，要求在 `do_phase` 函数中调用 `myfunc(s)`，显示出自己的学号。

6.2.6 第 5 关复位位表的修改

修改二进制可复位目标文件 `phase5.o` 的复位位节中的内容（不允许修改代码节和数据节），使其与 `main.o` 链接后，生成的执行程序运行时，显示 `Class Name : Computer Foundation. Teacher Name : Xu Xiangyang`。

6.2.7 第 6 关强弱符号

不准修改 `main.c` 和 `phase6.o`，通过增补一个文件，使得程序链接后，能够输出自己的学号。

```
gcc -no-pie -o linkbomb6 main.o phase6.o phase6_patch.o
```

6.2.8 第 7 关只读数据节的修改

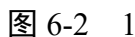
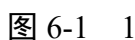
修改 `phase7.o` 中只读数据节（不准修改代码节），使其与 `main.o` 链接后，能够输出自己的学号。

6.3 实验记录及问题回答

6.3.1 第 1 关数据节的修改

这里我们用下好的 `hexedit`，然后打开二进制查看，之后在 `abcde...` 的地方改为我们的学号就好了

,



44

6.3.3 第3关有参数的函数调用的机器指令修改

我们来破解 Phase 3。这一关的目标是让 do_phase 调用 myfunc(pos)，这涉及到参数传递。

我们需要在 0x33 的位置写入两步操作：

准备参数：把保存在堆栈里的 pos 取出来，重新放回 %edi 寄存器（这是 myfunc 要读取的地方）。。

调用函数：执行 call myfunc。

Listing 12 phase3

```
1      phase3.o:   file format elf64-x86-64
2
3
4      Disassembly of section .text:
5
6      0000000000000000 <myfunc>:
7      0:  f3 0f 1e fa      endbr64
8      4:  55              push  %rbp
9      5:  48 89 e5        mov   %rsp,%rbp
10     8:  48 83 ec 10     sub   $0x10,%rsp
11     c:  89 7d fc        mov   %edi,-0x4(%rbp)
12     f:  8b 45 fc        mov   -0x4(%rbp),%eax
13    12:  89 c6          mov   %eax,%esi
14    14:  48 8d 3d 00 00 00 00 lea   0x0(%rip),%rdi    # 1b <myfunc+0x1b>
15    17: R_X86_64_PC32    .rodata-0x4
16    1b:  b8 00 00 00 00    mov   $0x0,%eax
17    20:  e8 00 00 00 00    call  25 <myfunc+0x25>
18    21: R_X86_64_PLT32    printf-0x4
19    25:  90              nop
20    26:  c9              leave
21    27:  c3              ret
22
23    0000000000000028 <do_phase>:
24    28:  f3 0f 1e fa      endbr64
25    2c:  55              push  %rbp
26    2d:  48 89 e5        mov   %rsp,%rbp
27    30:  89 7d fc        mov   %edi,-0x4(%rbp)
28    33:  90              nop
29    34:  90              nop
30    35:  90              nop
```


华中科技大学课程实验报告

```
5
6      0000000000000000 <myfunc>:
7      0:  f3 0f 1e fa      endbr64
8      4:  55              push  %rbp
9      5:  48 89 e5          mov   %rsp,%rbp
10     8:  48 83 ec 10       sub   $0x10,%rsp
11     c:  48 89 7d f8      mov   %rdi,-0x8(%rbp)
12    10:  48 8b 45 f8      mov   -0x8(%rbp),%rax
13    14:  48 89 c6          mov   %rax,%rsi
14    17:  48 8d 3d 00 00 00 00 lea   0x0(%rip),%rdi    # 1e <myfunc+0x1e>
15    1a: R_X86_64_PC32     .rodata-0x4
16    1e:  b8 00 00 00 00     mov   $0x0,%eax
17    23:  e8 00 00 00 00     call  28 <myfunc+0x28>
18    24: R_X86_64_PLT32     printf-0x4
19    28:  90                nop
20    29:  c9                leave
21    2a:  c3                ret
22
23     000000000000002b <do_phase>:
24    2b:  f3 0f 1e fa      endbr64
25    2f:  55              push  %rbp
26    30:  48 89 e5          mov   %rsp,%rbp
27    33:  48 83 ec 30       sub   $0x30,%rsp
28    37:  89 7d dc          mov   %edi,-0x24(%rbp)
29    3a:  64 48 8b 04 25 28 00 mov   %fs:0x28,%rax
30    41:  00 00
31    43:  48 89 45 f8      mov   %rax,-0x8(%rbp)
32    47:  31 c0            xor   %eax,%eax
33    49:  48 b8 55 32 30 32 32 movabs $0x3332313232303255,%rax
34    50:  31 32 33
35    53:  48 89 45 ed      mov   %rax,-0x13(%rbp)
36    57:  66 c7 45 f5 34 35 movw   $0x3534,-0xb(%rbp)
37    5d:  c6 45 f7 00      movb   $0x0,-0x9(%rbp)
38    61:  90                nop
39    62:  90                nop
40    63:  90                nop
41    64:  90                nop
42    65:  90                nop
43    66:  90                nop
44    67:  90                nop
45    68:  90                nop
```

```
46          69:  90                nop
47          6a:  90                nop
48          6b:  90                nop
49          6c:  90                nop
50          6d:  90                nop
51          6e:  90                nop
52          6f:  90                nop
53          70:  90                nop
54          71:  90                nop
55          72:  90                nop
56          73:  90                nop
57          74:  90                nop
58          75:  48 8b 45 f8      mov    -0x8(%rbp),%rax
59          79:  64 48 33 04 25 28 00 xor    %fs:0x28,%rax
60          80:  00 00
61          82:  74 05            je     89 <do_phase+0x5e>
62          84:  e8 00 00 00 00    call  89 <do_phase+0x5e>
63          85:  R_X86_64_PLT32    __stack_chk_fail-0x4
64          89:  c9                leave
65          8a:  c3                ret
```

我们需要在 0x61 处写入以下逻辑：

计算学号地址：把 `%rbp - 0x13` 的有效地址 (Effective Address) 加载到 `%rdi` 寄存器中。

指令： `lea -0x13(%rbp), %rdi`

调用函数： `call myfunc`。

找到 `do_phase` 里的特征序列： `C6 45 F7 00 90 90...`

从第一个 90 开始，依次输入：

`48 8D 7D ED E8 96 FF FF FF`

```
unwanhin@HusterUWH:~/Document/CodingIDE/VScode/Hust/csapp_lab/task5$ ./linkbomb4
please input you stuid : U202490035
gate 4: your ID is : U202212345!
Bye Bye !
```

图 6-8 4

6.3.5 第 5 关复位位表的修改

预期输出： `Class Name : Computer Foundation. Teacher Name : Xu Xiangyang`

Listing 14 phase5

华中科技大学课程实验报告

```
1      phase5.o:  file format elf64-x86-64
2
3
4      Disassembly of section .text:
5
6      0000000000000000 <do_phase>:
7      0:  f3 0f 1e fa      endbr64
8      4:  55              push  %rbp
9      5:  48 89 e5         mov   %rsp,%rbp
10     8:  48 83 ec 10      sub   $0x10,%rsp
11     c:  89 7d fc         mov   %edi,-0x4(%rbp)
12     f:  48 8d 35 00 00 00 00 lea    0x0(%rip),%rsi    # 16 <do_phase+0x16>
13    12: R_X86_64_PC32    originalclass-0x4
14    16:  48 8d 3d 00 00 00 00 lea    0x0(%rip),%rdi    # 1d <do_phase+0x1d>
15    19: R_X86_64_PC32    .rodata-0x4
16    1d:  b8 00 00 00 00      mov   $0x0,%eax
17    22:  e8 00 00 00 00      call  27 <do_phase+0x27>
18    23: R_X86_64_PLT32    printf-0x4
19    27:  48 8d 35 00 00 00 00 lea    0x0(%rip),%rsi    # 2e <do_phase+0x2e>
20    2a: R_X86_64_PC32    originalteacher-0x4
21    2e:  48 8d 3d 00 00 00 00 lea    0x0(%rip),%rdi    # 35 <do_phase+0x35>
22    31: R_X86_64_PC32    .rodata+0xb
23    35:  b8 00 00 00 00      mov   $0x0,%eax
24    3a:  e8 00 00 00 00      call  3f <do_phase+0x3f>
25    3b: R_X86_64_PLT32    printf-0x4
26    3f:  90              nop
27    40:  90              nop
28    41:  90              nop
29    42:  90              nop
30    43:  90              nop
31    44:  90              nop
32    45:  90              nop
33    46:  90              nop
34    47:  90              nop
35    48:  90              nop
36    49:  90              nop
37    4a:  90              nop
38    4b:  90              nop
39    4c:  90              nop
40    4d:  90              nop
41    4e:  90              nop
```

42	4f: 90	nop
43	50: 90	nop
44	51: 90	nop
45	52: 90	nop
46	53: 90	nop
47	54: 90	nop
48	55: c9	leave
49	56: c3	ret

字符串位置分析正确字符串：

0x00: "Computer Foundation"

0x20: "Xu Xiangyang"

错误字符串：

0x40: "c programming" (对应符号 originalclass)

0x60: "ma" (对应符号 originalteacher)

复位位表.rela.text 在文件中的起始偏移量是 0x3f8。每一条记录占 24 字节 (Offset 8 字节 + Info 8 字节 + Addend 8 字节)。我们只需要修改 Addend。

对此我们只要把 408 和 450 的 FC 改成 BC 就能成功了

```
unwanhin@HusterUWH:~/Document/CodingIDE/VScode/Hust/csapp_lab/task5$ gcc no_pic.c
please input you stuid : U202490035
Class Name Computer Foundation
Teacher Name Xu Xiangyang
Bye Bye !
unwanhin@HusterUWH:~/Document/CodingIDE/VScode/Hust/csapp_lab/task5$
```

图 6-9 5

6.3.6 第 6 关强弱符号

我们需要知道 phase6.o 里哪个函数是「弱」的，或者哪个函数被调用但未定义。

```
unwanhin@HusterUWH:~/Document/CodingIDE/VScode/Hust/csapp_lab/task5$ objdump -t phase6.o

phase6.o:      file format elf64-x86-64

SYMBOL TABLE:
0000000000000000 l df *ABS* 0000000000000000 phase6.c
0000000000000000 l d .text 0000000000000000 .text
0000000000000000 l d .data 0000000000000000 .data
0000000000000000 l d .bss 0000000000000000 .bss
0000000000000000 l d .data.rel.local 0000000000000000 .data.rel.local
0000000000000000 l d .rodata 0000000000000000 .rodata
0000000000000000 l d .note.gnu-stack 0000000000000000 .note.gnu-stack
0000000000000000 l d .note.gnu.property 0000000000000000 .note.gnu.property
0000000000000000 l d .eh_frame 0000000000000000 .eh_frame
0000000000000000 l d .comment 0000000000000000 .comment
0000000000000000 g O .data.rel.local 0000000000000008 phase
0000000000000000 g F .text 000000000000003a do_phase
0000000000000008 O *COM* 0000000000000008 myprint
0000000000000000 *UND* 0000000000000000 _GLOBAL_OFFSET_TABLE_
0000000000000000 *UND* 0000000000000000 puts

unwanhin@HusterUWH:~/Document/CodingIDE/VScode/Hust/csapp_lab/task5$
```

图 6-10 6

发现:

符号: myprint

类型: O (Object, 表示它是一个变量, 不是函数)

位置: *COM* (Common Section)

大小: 8 (字节, 正好是一个 64 位指针的大小)

Listing 15 phase6

```
1      #include <stdio.h>
2
3      void print_my_id() {
4          printf("U202490035\n");
5      }
6
7      void (*myprint)() = print_my_id;
```

```
unwanhin@HusterUWH:~/Document/CodingIDE/VScode/Hust/csapp_lab/task5$ gcc -c phase6_patch.c -o phase6_patch.o
unwanhin@HusterUWH:~/Document/CodingIDE/VScode/Hust/csapp_lab/task5$ gcc -no-pie -o linkbomb6 main.o phase6.o phase6_patch.o
unwanhin@HusterUWH:~/Document/CodingIDE/VScode/Hust/csapp_lab/task5$ ./linkbomb6
please input you stuid : U202490035
U202490035
Bye Bye !
```

图 6-11 6

6.3.7 第 7 关只读数据节的修改

phase7.o 里硬编码了一个别人的学号

目标: 把它改成 U202490035。

```
unwanhin@HusterUWH:~/Document/CodingIDE/VScode/Hust/csapp_lab/task5$ objdump -s -j .rodata phase7.o
phase7.o:      file format elf64-x86-64

Contents of section .rodata:
0000 47617465 20373a20 55323032 32313233  Gate 7: U2022123
0010 343500          45.
unwanhin@HusterUWH:~/Document/CodingIDE/VScode/Hust/csapp_lab/task5$
```

图 6-12 7

我们像 phase1 那样改就好了

```
unwanhin@HusterUWH:~/Document/CodingIDE/VScode/Hust/csapp_lab/task5$ hexedit phase7.o
unwanhin@HusterUWH:~/Document/CodingIDE/VScode/Hust/csapp_lab/task5$ gcc -no-pie -o linkbomb7 main.o phase7.o
unwanhin@HusterUWH:~/Document/CodingIDE/VScode/Hust/csapp_lab/task5$ ./linkbomb7
please input you stuid : U202490035
Gate 7: U202490035
Bye Bye !
unwanhin@HusterUWH:~/Document/CodingIDE/VScode/Hust/csapp_lab/task5$
```

图 6-13 13

6.4 体会

本次「链接炸弹」实验通过对 ELF 可复位目标文件的逆向分析与直接修改，让我深入理解了链接器的工作原理以及计算器程序的二进制表示。

对 ELF 文件结构的立体认识以往对 .text（代码节）、.data（数据节）、.rodata（只读数据节）的理解仅停留在概念层面。通过 Phase 1 和 Phase 7 的实践，我直接使用十六进制编辑器修改了数据节和只读数据节的内容，直观地看到了变量与常量在二进制文件中的存储形式，理解了为什么 C 语言字符串需要截断以及 ASCII 码的底层映像。

机器级指令的编码与修改 Phase 2、3、4 是本次实验的难点，也是收获最大的部分。我学会了如何手动构造机器指令

复位位与符号解析的本质 Phase 5 和 Phase 6 展示了链接器最核心的功能。

复位位表：在 Phase 5 中，我没有修改一行代码，仅通过修改 .rela.text 中的 Addend（加数），就改变了指针引用的目标字符串。这让我明白了链接器是如何将代码中的占位符修正为最终的内存地址的。

7 ARM 指令系统的理解

7.1 实验目的与要求

通过在 ARM 虚拟环境下调试执行程序，了解 ARM 的指令系统。

实验环境：ARM 虚拟实验环境 QEMU

工具：gcc, gdb 等

7.2 实验内容

程序及操作方法见 <ARM 实验任务.pdf>

7.2.1 任务 1、C 与汇编的混合编程

7.2.2 任务 2、内存拷贝及优化实验

查阅《ISA_A64_xml_A_profile-2023-09.pdf》，指出 10 条不同指令（存数、取数、算术运算、转移指令、函数调用等都应覆盖）的功能。

7.3 实验记录及问题回答

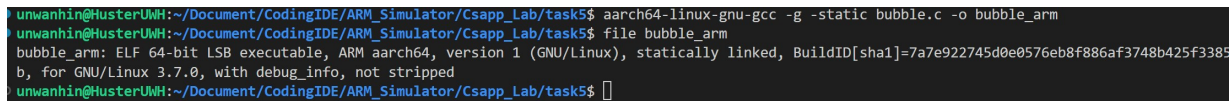
好的我们根据实验要求，自己先随便写一个冒泡排序的代码

Listing 16 task1.cpp

```
1      #include <stdio.h>
2      void print_array(int arr[], int n) {
3          for (int i = 0; i < n; i++) {
4              printf("%d ", arr[i]);
5          }
6          printf("\n");
7      }
8
9      int main() {
10         int arr[] = {64, 34, 25, 12, 22, 11, 90, 5};
11         int n = sizeof(arr) / sizeof(arr[0]);
12
13         printf("[ARM Experiment] Bubble Sort Demo\n");
14         printf("Original array: ");
```

```
15         print_array(arr, n);
16
17         int i, j, temp;
18         for (i = 0; i < n - 1; i++) {
19             for (j = 0; j < n - i - 1; j++) {
20                 if (arr[j] > arr[j + 1]) {
21                     temp = arr[j];
22                     arr[j] = arr[j + 1];
23                     arr[j + 1] = temp;
24                 }
25             }
26         }
27
28         printf("Sorted array: ");
29         print_array(arr, n);
30         return 0;
31     }
```

我们先是生成 arm 文件 `aarch64-linux-gnu-gcc -g -static bubble.c -o bubble_arm`，然后 `file bubble_arm`

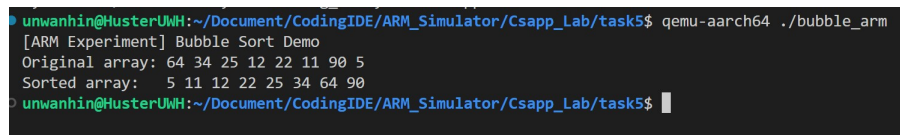


```
unwanhin@HusterUWH:~/Document/CodingIDE/ARM_Simulator/Csapp_Lab/task5$ aarch64-linux-gnu-gcc -g -static bubble.c -o bubble_arm
unwanhin@HusterUWH:~/Document/CodingIDE/ARM_Simulator/Csapp_Lab/task5$ file bubble_arm
bubble_arm: ELF 64-bit LSB executable, ARM aarch64, version 1 (GNU/Linux), statically linked, BuildID[sha1]=7a7e922745d0e0576eb8f886af3748b425f3385b, for GNU/Linux 3.7.0, with debug_info, not stripped
unwanhin@HusterUWH:~/Document/CodingIDE/ARM_Simulator/Csapp_Lab/task5$
```

图 7-1 arm

从 `file` 命令输出可以看到，生成的文件是 ELF 64-bit LSB executable, ARM aarch64，说明成功生成了 ARM 架构的可执行文件，无法在 x86 机器上直接运行。因此我们需要 arm 模拟工具

`qemu-aarch64 ./bubble_arm`



```
unwanhin@HusterUWH:~/Document/CodingIDE/ARM_Simulator/Csapp_Lab/task5$ qemu-aarch64 ./bubble_arm
[ARM Experiment] Bubble Sort Demo
Original array: 64 34 25 12 22 11 90 5
Sorted array: 5 11 12 22 25 34 64 90
unwanhin@HusterUWH:~/Document/CodingIDE/ARM_Simulator/Csapp_Lab/task5$
```

图 7-2 arm

程序正确执行，数组被成功排序，证明交叉编译与模拟环境工作正常。

```
unwanhin@HusterUWH:~/Document/CodingIDE/ARM_Simulator/Csapp_Lab/task5$ aarch64-linux-gnu-objdump -d bubble_arm | grep -A 20 "<main>:"
00000000040081c <main>:
40081c: d10143ff      sub     sp, sp, #0x50
400820: a9047bfd      stp     x29, x30, [sp, #64]
400824: 910103fd      add     x29, sp, #0x40
400828: f0000460      adrp    x0, 48f000 <tunable_list+0x880>
40082c: f9462c00      ldr     x0, [x0, #3160]
400830: f9400001      ldr     x1, [x0]
400834: f9001fe1      str     x1, [sp, #56]
400838: d2800001      mov     x1, #0x0 // #0
40083c: 900002a0      adrp    x0, 454000 <__gcc_personality_v0+0xac>
400840: 91116001      add     x1, x0, #0x458
400844: 910063e0      add     x0, sp, #0x18
400848: ad400420      ldp     q0, q1, [x1]
40084c: ad000400      stp     q0, q1, [x0]
400850: 52800100      mov     w0, #0x8 // #8
400854: b90013e0      str     w0, [sp, #16]
400858: 900002a0      adrp    x0, 454000 <__gcc_personality_v0+0xac>
40085c: 91100000      add     x0, x0, #0x400
400860: 940014d0      bl      405ba0 <_IO_puts>
400864: 900002a0      adrp    x0, 454000 <__gcc_personality_v0+0xac>
400868: 9110a000      add     x0, x0, #0x428
```

图 7-3 arm

观察汇编代码，可以看到与 x86 显著不同的特征：

寄存器使用 x29, x30（分别对应 FP 和 LR）以及 w0, w1 等通用寄存器。

大量使用 ldr 和 str 指令。这是因为 ARM 是 RISC 架构，运算不能直接在内存中进行，必须先加载到寄存器，运算后再存回内存。

指令长度固定（每行指令都是 4 字节），这也是 RISC 架构的典型特征。

7.3.1 指令及功能说明

1) LDR (Load Register)

指令格式：LDR <Xt>, [<Xn|SP>, <simmm>]

功能：从内存加载数据到寄存器。该指令以 Xn 或栈指针 SP 为基地址，加上可选的立即数偏移量 <simmm> 计算出目标内存地址，读取 64 位数据存入通用寄存器 Xt。这是最基础的取数指令。

2) STR (Store Register)

指令格式：STR <Xt>, [<Xn|SP>, <simmm>]

功能：将寄存器的数据存储在内存。该指令将通用寄存器 Xt 中的 64 位数据写入到以 Xn 或 SP 为基址加上偏移量 <simmm> 所指向的内存地址中。这是最基础的存数指令。

3) MOV (Move)

指令格式：MOV <Xd>, <Xm>

功能：寄存器间的数据传送。将源寄存器 X_m 的值复制到目标寄存器 X_d 。它实际上是 ORR（逻辑或）指令的一个别名（alias），即 ORR X_d, XZR, X_m ，但在汇编中常作为独立指令使用。

4) ADD (Add)

指令格式：ADD $\langle X_d|SP \rangle, \langle X_n|SP \rangle, \langle X_m \rangle, \langle \text{shift} \rangle \langle \text{amount} \rangle$

功能：加法运算。将寄存器 X_n 的值与寄存器 X_m （可经过移位操作）的值相加，结果写入目标寄存器 X_d 。常用于整数加法或地址计算。

5) SUB (Subtract)

指令格式：SUB $\langle X_d|SP \rangle, \langle X_n|SP \rangle, \langle \text{imm} \rangle$

功能：减法运算。将寄存器 X_n 的值减去立即数 $\langle \text{imm} \rangle$ ，结果写入目标寄存器 X_d 。也可用于寄存器之间的减法。

6) CMP (Compare)

指令格式：CMP $\langle X_n|SP \rangle, \langle X_m \rangle, \langle \text{shift} \rangle \langle \text{amount} \rangle$

功能：比较指令。实际上是设置标志位的减法指令（SUBS），将 X_n 减去 X_m ，结果不保存，但会根据结果更新程序状态寄存器（PSTATE）中的条件标志位（如 N, Z, C, V），常用于分支跳转前的条件判断。

7) AND (Bitwise AND)

指令格式：AND $\langle X_d|SP \rangle, \langle X_n \rangle, \langle X_m \rangle, \langle \text{shift} \rangle \langle \text{amount} \rangle$

功能：按位与运算。对寄存器 X_n 和 X_m 的值进行按位与操作，结果存入 X_d 。常用于掩码操作，提取特定位的值。

8) LSL (Logical Shift Left)

指令格式：LSL $\langle X_d \rangle, \langle X_n \rangle, \langle \text{imm} \rangle$

功能：逻辑左移。将寄存器 X_n 中的位向左移动 $\langle \text{imm} \rangle$ 指定的位数，低位补 0，结果存入 X_d 。这是 UBFM（无符号位域移动）指令的别名，常用于乘法（乘 2 的幂）或位操作。

9) B (Branch)

指令格式：B $\langle \text{label} \rangle$

功能：无条件跳转。程序无条件跳转到标签 $\langle \text{label} \rangle$ 指定的地址继续执行。这是实现循环或函数内部跳转的基本指令。条件跳转如 B.EQ（相等跳转）、B.NE（不等跳转）也是基于此指令的变体。

10) BL (Branch with Link)

指令格式: BL <label>

功能: 带链接跳转 (函数调用)。跳转到 <label> 指定的子程序地址执行, 同时将下一条指令的地址 (返回地址) 保存到链接寄存器 X30 (LR) 中。这是实现函数调用的核心指令, 子程序执行完毕后通过 RET 指令返回。

7.4 体会

本次实验是我首次脱离熟悉的 x86 架构, 进入 ARMv8 (AArch64) 体系结构的开发领域。通过搭建交叉编译环境并分析底层汇编代码, 我对计算器体系结构有了更立体的认知

1. 交叉编译

在实验中, 由于我的开发环境是 x86 架构, 无法直接生成或运行 ARM 指令。通过使用 aarch64-linux-gnu-gcc 进行交叉编译, 我生成了 ARM 格式的 ELF 文件; 而 qemu-aarch64 则通过仿真, 动态将 ARM 指令翻译为 x86 指令执行。这让我理解了嵌入式开发中「在高性能 PC 上编译, 在低功耗设备上运行」的标准工作流。

2. RISC 与 CISC 架构的差异通过反汇编冒泡排序程序, 我直观地观察到了 ARM (RISC) 与 x86 (CISC) 的显著不同

Load/Store 架构: 在 x86 中, 我可以写 `add [rax], 1` 直接修改内存; 但在 ARM 中, 我必须先用 `ldr` 将数据加载到寄存器, 运算后再用 `str` 写回。这虽然增加了指令条数, 但也让硬件设计更简单、流水线更高效。

3. 指令集的规整性相比于 x86 变长且复杂的指令格式, ARM64 的汇编代码看起来更加规整。函数调用约定 (Calling Convention) 也非常清晰, 前 8 个参数直接通过 x0-x7 传递, 返回值存在 x0, 这让汇编代码的阅读和分析变得相对容易。

8 附录

8.1 实验一源码

8.1.1 任务一

Listing 17 task1.cpp

```
1      #include <cstdio>
2      #include <cstring>
3      #include <vector>
4      #include <iostream>
5
6      #pragma pack(1)
7
8      const int N = 5;
9      const int N1 = 2;
10     const int N2 = 3;
11
12     struct student {
13         char name[8];
14         short age;
15         float score;
16         char remark[200];
17     };
18
19     student old_s[N], new_s[N];
20
21     int pack_student_bytebybyte(student* s, int sno, char*& buf);
22     int pack_student_whole(student* s, int sno, char*& buf);
23     int restore_student(char* buf, int len, student* s);
24
25     void print_student(student* s, int sno) {
26         printf("%-9s %-5s %-7s %-32s\n", "name", "age", "score", "remark");
27         for (int i = 0; i < sno; ++i) {
28             printf("%-9s %-5hd %-7.1f %-32s\n", s[i].name, s[i].age,
29                 s[i].score, s[i].remark);
30         }
31
32     int scan_student(student* s, int sno) {
```

60

```
71
72         // 3. 处理评语
73         for (int k = 0; k < 200; ++k) {
74             if (p[k] != 0) {
75                 aux.push_back(p[k]);
76             } else {
77                 break;
78             }
79         }
80         aux.push_back(0);
81     }
82
83     int len = aux.size();
84     buf = new char[len];
85     memcpy(buf, aux.data(), len);
86     return len;
87 }
88
89
90 int pack_student_whole(student* s, int sno, char*& buf) {
91     int len = 0;
92     for (int i = 0; i < sno; ++i) {
93         len += strlen(s[i].name) + 1;
94         len += 1 + ((s[i].age & 0xFF00) ? 2 : 1);
95
96         len += sizeof(float); // float 不压缩
97         len += strlen(s[i].remark) + 1;
98     }
99
100    char* temp_buf = new char[sizeof(student) * sno];
101    char* p = temp_buf;
102
103    for (int i = 0; i < sno; ++i) {
104        // 姓名
105        strcpy(p, s[i].name);
106        p += strlen(s[i].name) + 1;
107
108        // 年龄 (只拷贝低8位)
109        memcpy(p, &s[i].age, 1);
110        p++;
111    }
```



```
112         // 分数
113         memcpy(p, &s[i].score, sizeof(float));
114         p += sizeof(float);
115
116         // 评语
117         strcpy(p, s[i].remark);
118         p += strlen(s[i].remark) + 1;
119     }
120
121     int real_len = p - temp_buf;
122     buf = new char[real_len];
123     memcpy(buf, temp_buf, real_len);
124     delete[] temp_buf;
125
126     return real_len;
127 }
128
129 int restore_student(char* buf, int len, student* s) {
130     char* p = buf;
131     int sno = 0;
132
133     while (p < (buf + len)) {
134         // 1. 还原姓名
135         char* pname = s[sno].name;
136         int char_count = 0;
137         while (*p != '\0') {
138             if(char_count < 7) {
139                 *(pname++) = *(p++);
140                 char_count++;
141             } else {
142                 p++;
143             }
144         }
145         *pname = '\0';
146         memset(pname + 1, 0, 7 - char_count);
147
148         p++; // 跳过压缩数据中的 \0
149
150         // 2. 还原年龄 (取1字节放到低位, 高位补0)
151         char* p_age = (char*)&s[sno].age;
152         p_age[0] = *(p++);
```

```
153         p_age[1] = 0;
154
155         // 3. 还原分数 (直接拷贝4字节)
156         char* p_score = (char*)&s[sno].score;
157         for (int k = 0; k < sizeof(float); ++k) {
158             p_score[k] = *(p++);
159         }
160
161         // 4. 还原评语
162         char* p_remark = s[sno].remark;
163         while (*p != '\0') {
164             *(p_remark++) = *(p++);
165         }
166         *p_remark = '\0';
167         p++;
168
169         sno++;
170     }
171     return sno;
172 }
173
174 int main() {
175
176     int raw_size = scan_student(old_s, N);
177
178     char *buf1 = nullptr, *buf2 = nullptr;
179
180     int len1 = pack_student_bytebybyte(old_s, N1, buf1);
181
182     int len2 = pack_student_whole(old_s + N1, N2, buf2);
183
184     printf("\nCompressed size: %d bytes.\nCompress ratio: %.2f%%\n\n",
185         len1 + len2, 100.0 * (len1 + len2) / raw_size);
186
187     restore_student(buf1, len1, new_s);
188     restore_student(buf2, len2, new_s + N1);
189
190     printf("--- Restored Data ---\n");
191     print_student(new_s, N);
192
193     delete[] buf1;
```

```
194         delete[] buf2;
195         return 0;
196     }
```

8.1.2 任务二

Listing 18 task2.cpp

```
1
2     #include <stdio>
3     #include <stdlib> // for exit()
4
5     int absVal(int x) {
6         // 利用算术右移构造掩码: 正数为 0, 负数为 -1 (0xFFFFFFFF)
7         int mask = x >> 31;
8         // (x ^ 0) + 0 = x
9         // (x ^ -1) + 1 = ~x + 1 = -x
10        return (x ^ mask) + (mask & 1);
11    }
12
13    int negate(int x) {
14        return ~x + 1;
15    }
16
17    int bitAnd(int x, int y) {
18        return ~(~x | ~y); // 德摩根定律
19    }
20
21    int bitor_func(int x, int y) {
22        return ~(~x & ~y); // 德摩根定律
23    }
24
25    int bitXor(int x, int y) {
26        //  $x \sim y = (x \oplus \sim y) \mid (\sim x \oplus y)$ 
27        // 再次应用德摩根定律转为仅用  $\oplus$  和  $\sim$ 
28        return ~(~(x & ~y) & ~(~x & y));
29    }
30
31    int isTmax(int x) {
32        // 如果 x 是 0x7FFFFFFF, 那么  $x \sim 0x7FFFFFFF$  结果为 0
33        // !0 = 1
34        return !(x ^ 0x7FFFFFFF);
35    }
36
37    int bitCount(int x) {
38        // 分治法 (Parallel bit counting)
39        int mask1 = 0x55555555;
```

```
34         int mask2 = 0x33333333;
35         int mask4 = 0x0F0F0F0F;
36         int mask8 = 0x00FF00FF;
37         int mask16 = 0x0000FFFF;
38
39         x = (x & mask1) + ((x >> 1) & mask1);
40         x = (x & mask2) + ((x >> 2) & mask2);
41         x = (x & mask4) + ((x >> 4) & mask4);
42         x = (x & mask8) + ((x >> 8) & mask8);
43         x = (x & mask16) + ((x >> 16) & mask16);
44         return x;
45     }
46     int bitMask(int highbit, int lowbit) {
47         return (1 << (highbit + 1)) + ~(1 << lowbit) + 1;
48     }
49
50     int addOK(int x, int y) {
51         int sum = x + y;
52         int sign_x = (x >> 31) & 1;
53         int sign_y = (y >> 31) & 1;
54         int sign_sum = (sum >> 31) & 1;
55         return (!(sign_x ^ sign_y)) & (sign_x ^ sign_sum);
56     }
57
58     int byteSwap(int x, int n, int m) {
59         int n_val = (x >> (n * 8)) & 0xFF;
60         int m_val = (x >> (m * 8)) & 0xFF;
61
62         // 2. 构造掩码将原位置清零 (Mask out)
63         int mask = (0xFF << (n * 8)) | (0xFF << (m * 8));
64         x = x & ~mask;
65
66         x = x | (n_val << (m * 8)) | (m_val << (n * 8));
67
68         return x;
69     }
70
71     int bang(int x) {
72         return ((x | (~x + 1)) >> 31) + 1;
73     }
74
```

```
75     int bitParity(int x) {
76         x ^= x >> 16;
77         x ^= x >> 8;
78         x ^= x >> 4;
79         x ^= x >> 2;
80         x ^= x >> 1;
81         return x & 1;
82     }
83
84     int absVal_standard(int x) { return (x < 0) ? -x : x; }
85     int negate_standard(int x) { return -x; }
86     int bitAnd_standard(int x, int y) { return x & y; }
87     int bitOr_standard(int x, int y) { return x | y; }
88     int bitXor_standard(int x, int y) { return x ^ y; }
89     int isTmax_standard(int x) { return x == 0x7FFFFFFF; }
90     int bitCount_standard(int x) {
91         int cnt = 0;
92         unsigned int ux = (unsigned int)x; // 转无符号防止右移补1
93         while(ux) { if(ux & 1) cnt++; ux >>= 1; }
94         return cnt;
95     }
96     int bitMask_standard(int highbit, int lowbit) {
97         int x = 0;
98         for (int i = lowbit; i <= highbit; ++i) x |= (1 << i);
99         return x;
100    }
101    int addOK_standard(int x, int y) {
102        long long sum = (long long)x + y;
103        return (sum > 2147483647 || sum < -2147483648);
104    }
105    int byteSwap_standard(int x, int n, int m) {
106        unsigned char* p = (unsigned char*)&x;
107        unsigned char tmp = p[n];
108        p[n] = p[m];
109        p[m] = tmp;
110        return x;
111    }
112    int bang_standard(int x) { return !x; }
113    int bitParity_standard(int x) {
114        int cnt = 0;
115        unsigned int ux = (unsigned int)x;
```

华中科技大学课程实验报告

```
116         while(ux) { cnt ^= (ux & 1); ux >>= 1; }
117         return cnt;
118     }
119
120     void run_test(const char* func_name, int my_res, int std_res) {
121         if (my_res == std_res) {
122             printf("[PASS] %-25s | Result: %10d (0x%08X)\n", func_name,
123                    my_res, my_res);
124         } else {
125             printf("[FAIL] %-25s | My: %d, Std: %d\n", func_name,
126                    my_res, std_res);
127             // exit(1);
128         }
129     }
130
131     int main() {
132
133         printf("===== Batch Test Report\n");
134         printf("%-32s | %-20s\n", "Function & Input", "Output Status");
135         printf("-----\n");
136
137         // 1. absVal
138         run_test("absVal(8)", absVal(8), absVal_standard(8));
139         run_test("absVal(-8)", absVal(-8), absVal_standard(-8));
140
141         // 2. negate
142         run_test("negate(50)", negate(50), negate_standard(50));
143         run_test("negate(-50)", negate(-50), negate_standard(-50));
144
145         // 3. bitAnd
146         run_test("bitAnd(0xAA, 0x55)", bitAnd(0xAA, 0x55),
147                bitAnd_standard(0xAA, 0x55));
148
149         // 4. bitOr
150         run_test("bitor_func(0xAA, 0x55)", bitor_func(0xAA, 0x55),
151                bitor_standard(0xAA, 0x55));
152
153         // 5. bitXor
154         run_test("bitXor(0xAA, 0x55)", bitXor(0xAA, 0x55),
155                bitXor_standard(0xAA, 0x55));
```

华中科技大学课程实验报告

```
151
152      // 6. isTmax
153      run_test("isTmax(0x7FFFFFFF)", isTmax(0x7FFFFFFF),
154              isTmax_standard(0x7FFFFFFF));
155
156      run_test("isTmax(0x12345678)", isTmax(0x12345678),
157              isTmax_standard(0x12345678));
158
159      // 7. bitCount
160      run_test("bitCount(0x12345678)", bitCount(0x12345678),
161              bitCount_standard(0x12345678));
162      run_test("bitCount(-1)", bitCount(-1), bitCount_standard(-1));
163
164      // 8. bitMask
165      run_test("bitMask(5, 3)", bitMask(5, 3), bitMask_standard(5, 3));
166      run_test("bitMask(10, 0)", bitMask(10, 0), bitMask_standard(10,
167              0));
168
169      // 9. addOK
170      // 0x7FFFFFFF + 1 = 溢出 -> 预期返回 1
171      run_test("addOK(MAX, 1) [Overflow]", addOK(0x7FFFFFFF, 1),
172              addOK_standard(0x7FFFFFFF, 1));
173      run_test("addOK(1, 1) [Normal]", addOK(1, 1), addOK_standard(1,
174              1));
175
176      // 10. byteSwap
177      run_test("byteSwap(0x12345678,0,3)", byteSwap(0x12345678, 0, 3),
178              byteSwap_standard(0x12345678, 0, 3));
179      run_test("byteSwap(0xDEADBEEF,1,2)", byteSwap(0xDEADBEEF, 1, 2),
180              byteSwap_standard(0xDEADBEEF, 1, 2));
181
182      // 11. bang
183      run_test("bang(3)", bang(3), bang_standard(3));
184      run_test("bang(0)", bang(0), bang_standard(0));
185
186      // 12. bitParity
187      run_test("bitParity(5) [0101]", bitParity(5),
188              bitParity_standard(5));
189      run_test("bitParity(7) [0111]", bitParity(7),
190              bitParity_standard(7));
191
192      printf("-----\n");
```

```
182         printf("Summary: All Logic Gates Simulated Successfully.\n");
183         printf("=====\n");
184
185         return 0;
186     }
```

8.2 实验二源码

8.2.1 任务一

Listing 19 task1.cpp

```
1
2     #define _CRT_SECURE_NO_WARNINGS
3     #include <iostream>
4     #include <vector>
5     #include <iomanip> // 用于格式化输出
6     #include <cstdlib>
7     #include <ctime>
8     #include <cstring>
9     #include <cstdio>
10    #include <chrono> // 高精度计时
11
12    using namespace std;
13    #pragma pack(1)
14    struct Student {
15        char name[12];    // 8 bytes
16        char sid[11];     // 11 bytes
17        short scores[8]; // 16 bytes
18        short average;   // 2 bytes
19    };
20    #pragma pack()
21
22    const int N = 30;
23    void InitData(vector<Student>& students) {
24        strcpy_s(students[0].name, "阮云轩");
25        strcpy_s(students[0].sid, "U202490035");
26        for (int k = 0; k < 8; ++k) students[0].scores[k] = 95;
27        students[0].average = 0;
28
29        srand((unsigned)time(NULL));
```



```
30         for (int i = 1; i < N; ++i) {
31             sprintf_s(students[i].name, "S_%d", i);
32             sprintf_s(students[i].sid, "U2024%05d", i);
33
34
35             for (int k = 0; k < 8; ++k) {
36                 students[i].scores[k] = 50 + rand() % 51;
37             }
38             students[i].average = 0;
39         }
40     }
41     void CalcAvg_C(Student* s, int count) {
42         for (int i = 0; i < count; ++i) {
43             int sum = 0;
44             for (int k = 0; k < 8; ++k) {
45                 sum += s[i].scores[k];
46             }
47
48             s[i].average = (short)(sum >> 3);
49         }
50
51     }
52     void Sort_C(Student* s, int count) {
53         for (int i = 0; i < count - 1; ++i) {
54             bool swapped = false;
55             for (int j = 0; j < count - 1 - i; ++j) {
56                 if (s[j].average < s[j + 1].average) {
57                     Student temp = s[j];
58                     s[j] = s[j + 1];
59                     s[j + 1] = temp;
60                     swapped = true;
61                 }
62             }
63             if (!swapped) break;
64         }
65
66     }
67
68     void PrintStudents(const vector<Student>& s) {
69         cout << left << setw(10) << "Name"
70             << setw(12) << "SID"
```

华中科技大学课程实验报告

```
71         << setw(30) << "Scores (1-8)"
72         << "Avg" << endl;
73         cout <<
74             "-----"
75             << endl;
76
77         for (int i = 0; i < N; ++i) {
78             cout << left << setw(10) << s[i].name
79             << setw(12) << s[i].sid << " ";
80
81             for (int k = 0; k < 8; ++k) {
82                 cout << s[i].scores[k] << " ";
83             }
84             cout << " | " << s[i].average << endl;
85         }
86         cout << endl;
87     }
88
89     int main() {
90         vector<Student> students(N);
91         InitData(students);
92         cout << ">>> Task 1: Student Management System (N=" << N << ")" <<
93             endl << endl;
94         auto start = chrono::high_resolution_clock::now();
95         CalcAvg_C(students.data(), N);
96         auto end = chrono::high_resolution_clock::now();
97         double timeAvg = chrono::duration_cast<chrono::nanoseconds>(end -
98             start).count() / 1000.0;
99         cout << "[Timing] Calculate Average Cost: " << timeAvg << " us
100             (microseconds)" << endl;
101         cout << "\n--- Before Sort (First 5) ---" << endl;
102         for (int i = 0; i < 5; i++) cout << students[i].name << " Avg: "
103             << students[i].average << endl;
104         start = chrono::high_resolution_clock::now();
105         Sort_C(students.data(), N);
106         end = chrono::high_resolution_clock::now();
107         double timeSort = chrono::duration_cast<chrono::nanoseconds>(end -
108             start).count() / 1000.0;
109         cout << "[Timing] Sort Cost: " << timeSort << " us
110             (microseconds)" << endl;
111         cout << "\n--- Final Result (All Students Sorted) ---" << endl;
```

```
104         PrintStudents(students);
105
106         return 0;
107     }
```

8.2.2 任务二

Listing 20 task2.asm

```
1      .686P
2      .model flat, C
3
4      OFFSET_SCORES equ 19
5      OFFSET_AVG     equ 35
6      STRUCT_SIZE equ 37
7
8      .code
9
10     CalcAvg_ASM PROC
11     push ebp
12     mov ebp, esp
13     push ebx
14     push esi
15     push edi
16
17     mov esi, [ebp + 8]
18     mov ecx, [ebp + 12]
19     test ecx, ecx
20     jz _Exit
21
22     _Loop_Student:
23     push ecx
24
25     ; --- 计算总分 ---
26     xor eax, eax
27     lea ebx, [esi + OFFSET_SCORES]
28     mov ecx, 8
29
30     _Loop_Score:
31     movsx edx, word ptr [ebx]
32     add eax, edx
```

```
33         add ebx, 2
34         dec ecx
35         jnz _Loop_Score
36         sar eax, 3          ; EAX = average
37
38         mov word ptr [esi + OFFSET_AVG], ax
39
40         add esi, STRUCT_SIZE
41
42         pop ecx
43         dec ecx
44         jnz _Loop_Student
45
46         _Exit:
47         pop edi
48         pop esi
49         pop ebx
50         pop ebp
51         ret
52     CalcAvg_ASM ENDP
53
54     END
```

Listing 21 task2.cpp

```
1
2     #define _CRT_SECURE_NO_WARNINGS
3
4     #include <iostream>
5     #include <vector>
6     #include <iomanip>
7     #include <cstdlib>
8     #include <ctime>
9     #include <cstring>
10    #include <cstdio>
11    #include <chrono>
12
13    using namespace std;
14
15    #pragma pack(1)
16    struct Student {
17        char name[8];    // 8 bytes
```

华中科技大学课程实验报告

```
18         char sid[11];    // 11 bytes
19         short scores[8]; // 16 bytes
20         short average;    // 2 bytes
21     };
22     #pragma pack()
23     const int N = 5000;
24     extern "C" void Sort_ASM_Opt(Student* s, int count);
25
26     void Helper_CalcAvg(Student* s, int count) {
27         for (int i = 0; i < count; ++i) {
28             int sum = 0;
29             for (int k = 0; k < 8; ++k) sum += s[i].scores[k];
30             s[i].average = (short)(sum >> 3);
31         }
32     }
33
34     void InitData(vector<Student>& students) {
35         strcpy(students[0].name, "Ryan");
36         strcpy(students[0].sid, "U202490035");
37
38         srand((unsigned)time(NULL));
39         for (int i = 0; i < N; ++i) { // 全部随机生成
40             if (i > 0) {
41                 sprintf(students[i].name, "S_%d", i);
42                 sprintf(students[i].sid, "U202%05d", i);
43             }
44             for (int k = 0; k < 8; ++k) students[i].scores[k] = 50 +
45                 rand() % 51;
46             students[i].average = 0;
47         }
48     }
49
50     void PrintStudents(const vector<Student>& s) {
51         cout << left << setw(10) << "Name" << setw(12) << "SID" << "Avg"
52             << endl;
53         cout << "-----" << endl;
54         for (int i = 0; i < 10; ++i) { // 只打印前10个验证
55             cout << left << setw(10) << s[i].name
56                 << setw(12) << s[i].sid
57                 << s[i].average << endl;
```

```
57         cout << "... " << endl;
58     }
59
60     int main() {
61         vector<Student> students(N);
62         InitData(students);
63
64         Helper_CalcAvg(students.data(), N);
65
66         cout << ">>> Task 4: ASM Optimized Sort (SIMD Swap)" << endl;
67         cout << "Sorting " << N << " students..." << endl;
68
69         auto start = chrono::high_resolution_clock::now();
70
71         Sort_ASM_Opt(students.data(), N);
72
73         auto end = chrono::high_resolution_clock::now();
74         double timeSort = chrono::duration_cast<chrono::milliseconds>(end
75             - start).count();
76
77         cout << "[Timing] Optimized Sort Cost: " << timeSort << " ms" <<
78             endl;
79
80         cout << "\n--- Result (Top 10) ---" << endl;
81         PrintStudents(students);
82         return 0;
83     }
```

8.2.3 任务三

Listing 22 task3.asm

```
1     .686P
2     .model flat, C
3
4     OFFSET_SCORES equ 19
5     OFFSET_AVG    equ 35
6     STRUCT_SIZE   equ 37
7
8     .code
9
```

华中科技大学课程实验报告

```
10      CalcAvg_ASM_Opt PROC uses esi,
11      ptr_data:DWORD, ; [ebp+8]
12      count:DWORD    ; [ebp+12]
13
14      mov esi, ptr_data    ; ESI = 当前学生指针
15      mov ecx, count      ; ECX = 学生总数
16
17      test ecx, ecx
18      jz _Exit
19
20      _Loop_Student:
21
22
23      xor eax, eax        ; sum = 0
24      ; OFFSET_SCORES = 19
25
26      add ax, word ptr [esi + 19] ; 第1门课
27      add ax, word ptr [esi + 21] ; 第2门课
28      add ax, word ptr [esi + 23] ; 第3门课
29      add ax, word ptr [esi + 25] ; 第4门课
30      add ax, word ptr [esi + 27] ; 第5门课
31      add ax, word ptr [esi + 29] ; 第6门课
32      add ax, word ptr [esi + 31] ; 第7门课
33      add ax, word ptr [esi + 33] ; 第8门课
34
35      ; --- 计算平均分 ---
36      sar eax, 3          ; 右移 3 位 (除以 8)
37
38      ; --- 存回结构体 ---
39      mov word ptr [esi + OFFSET_AVG], ax
40
41      ; --- 下一个学生 ---
42      add esi, STRUCT_SIZE
43      dec ecx
44      jnz _Loop_Student
45
46      _Exit:
47      ret
48      CalcAvg_ASM_Opt ENDP
49
50      END
```

8.2.4 任务四

Listing 23 task4.asm

```
1      .686P
2      .xmm
3      .model flat, C
4
5      OFFSET_AVG equ 35
6      STRUCT_SIZE equ 37
7
8      .code
9      Sort_ASM_Opt PROC uses ebx esi edi,
10     ptr_data:DWORD, ; [ebp+8]
11     count:DWORD ; [ebp+12]
12
13     mov esi, ptr_data ; ESI = 数组首地址
14     mov ecx, count ; ECX = N
15     dec ecx ; 外层循环次数 N-1
16     js _Exit ; 若 N <= 0 退出
17
18     _Outer_Loop:
19     push ecx ; 保存外层计数
20     mov edi, esi ; EDI = 当前扫描指针
21     xor edx, edx ; EDX = swapped 标志 (0: false)
22
23     _Inner_Loop:
24     mov ax, word ptr [edi + OFFSET_AVG]
25     cmp ax, word ptr [edi + STRUCT_SIZE + OFFSET_AVG]
26     jge _No_Swap ; 如果 前 >= 后 (降序), 不需要交换
27
28
29     ; 1. 交换前 16 字节 (0-15)
30     movups xmm0, xmmword ptr [edi] ; 把 A 放入 xmm0
31     movups xmm1, xmmword ptr [edi + STRUCT_SIZE] ; 把 B 放入 xmm1
32     movups xmmword ptr [edi], xmm1 ; B -> A
33     movups xmmword ptr [edi + STRUCT_SIZE], xmm0 ; A -> B
34
35     ; 2. 交换中 16 字节 (16-31)
36     movups xmm0, xmmword ptr [edi + 16]
37     movups xmm1, xmmword ptr [edi + STRUCT_SIZE + 16]
38     movups xmmword ptr [edi + 16], xmm1
```


华中科技大学课程实验报告

```
39      movups xmmword ptr [edi + STRUCT_SIZE + 16], xmm0
40
41      ; 3. 交换最后 5 字节 (32-36)
42      ; 用普通寄存器搬运这最后一点点
43      mov eax, dword ptr [edi + 32]
44      mov ebx, dword ptr [edi + STRUCT_SIZE + 32]
45      mov dword ptr [edi + 32], ebx
46      mov dword ptr [edi + STRUCT_SIZE + 32], eax
47
48      mov al, byte ptr [edi + 36]
49      mov bl, byte ptr [edi + STRUCT_SIZE + 36]
50      mov byte ptr [edi + 36], bl
51      mov byte ptr [edi + STRUCT_SIZE + 36], al
52
53      mov edx, 1
54
55      _No_Swap:
56      add edi, STRUCT_SIZE ; 指针后移
57      dec ecx
58      jnz _Inner_Loop
59
60      pop ecx                ; 恢复外层计数
61
62      test edx, edx          ; 检查 swapped 是否为 0
63      jz _Exit               ; 如果这一轮完全没交换, 说明已经排好了, 直接退出!
64
65      dec ecx
66      jnz _Outer_Loop
67
68      _Exit:
69      ret
70      Sort_ASM_Opt ENDP
71
72      END
```